

THE MAKING OF OMNITOME

A Technical Memoir of Local-First Reading, Compression, and the Long Road to v1.0.2

The Omnitome Engineering Team

Omnitome Publishing

First Edition | October 2026

FIRST EDITION · OCTOBER 2026

Copyright

Copyright © October 2026, The Omnitome Engineering Team.

Published by **Omnitome Publishing**.

All rights reserved for this original manuscript, except where applicable law permits otherwise. The application discussed in this book has its own software licensing terms. As of the documented v1.0.2 release, the public Omnitome repository identifies its license as **AGPL-3.0-only**. That repository license is separate from the copyright status of this book and from the licenses of any books, research papers, fonts, models, or other materials that users import into Omnitome.

This memoir is an original account of a software project. Product names, model names, framework names, and other marks belong to their respective owners. Their mention is for description and technical context, not to imply endorsement.

The account is grounded in saved engineering session records, project retrospectives, repository documentation, source code, release notes, and project conversations available to the authors in October 2026. It is not a claim that every early experiment was preserved in a complete, independently reproducible audit trail. The manuscript distinguishes documented implementation details from recollection and retrospective interpretation wherever that difference matters.

Edition: First Edition

Publication date: October 2026

Dedication

For every reader who has loved a story enough to keep going, and every builder who has looked at a stubborn problem and thought: *there has to be a better way to do this without losing what made it worthwhile.*

A Note on Evidence, Memory, and the Shape of This Story

Software history is often told backward. A working product appears, its architecture looks inevitable, and the rough prototypes that preceded it become a clean staircase leading toward success. Real development is less tidy. The next step is usually unclear. A design decision may look obvious only after three alternatives have failed. A benchmark may reveal that a model was never doing what its name suggested. A feature may work in development, then fail when frozen inside an installer because a phonemizer expects a file that was never bundled.

This book tries not to polish those rough edges out of existence.

The narrative has four principal sources. First are the saved Smart Reader engineering-session logs, especially the records covering the semantic and frontend work in September 2026, the provenance and import phases, the GPU and parser investigations, the Omnitome rebrand, Kokoro text-to-speech, and the Tauri packaging sequence. Second are the dated personal session reports from September and October 2026. Third is the public repository, including its README, architecture and feature documentation, benchmark records, roadmap, session handoff, and implementation files. Fourth is project recollection preserved in the chat history, including the origin story of the late-2024 GPT4All summarizer experiments.

These sources do not have identical evidentiary weight. The October 22 handoff is explicit that some pre-release measurements and historical descriptions require re-verification. The repository's current README is useful for the release identity and architecture, but its headline compression ratio should not be mistaken for a universal measurement on every input. The early GPT4All experiment is part of the project's origin recollection, but the material available for this book does not preserve a full late-2024 commit-by-commit diary. This book therefore does not invent an exact launch day, model quantization, benchmark corpus, or experimental result for that first prototype.

Similarly, a recorded test pass demonstrates that a particular test suite passed in a particular recorded state. It does not prove that every possible workload is correct, every GPU has the same performance, or every future build will behave identically. Benchmark figures in these pages are identified as measurements from the project machine, not promises about arbitrary computers.

Where chronology is firm, the prose says so. Where a detail is a reasonable interpretation of the development record, it is presented as interpretation. Where the evidence does not support a specific claim, the book leaves that blank rather than filling it with a cinematic invention.

That discipline is itself part of the story. Omnitome was built around the idea that a useful derived reading should never pretend to be the original source. A memoir about the engineering of that product should follow the same rule: tell the story, but leave a path back to the evidence.

Contents

Prologue: The Four-Thousand-Chapter Problem 6

Part I: The Frustration Before the Architecture

1. The First Local Summarizer 8
2. The Product Hiding Inside the Prototype 10
3. Smart Reader Learns What It Is 12

Part II: Turning a Reading Idea into a System

4. Structure Before Intelligence 14
5. September: Rebuilding the Semantic Foundation 16
6. React, Routes, and the Shape of the Reader 18
7. Compression Is a Contract, Not a Prompt 20
8. Two Representations, One Trust Boundary 22
9. Import Became a Product of Its Own 24

Part III: Real Documents, Real Limits

10. Documents Refuse to Behave 26
11. The Four-Gigabyte Ceiling 28
12. Failure Must Be Visible 30

Part IV: The Library Gets a Name and a Sensory World

13. From Smart Reader to Omnitome 32
14. A Voice That Can Stay Offline 34
15. Sound as Interface 36

Part V: A Web Application Learns to Become a Desktop Application

16. Why Tauri Changed the Problem 38
17. The Installer Is Another Runtime 40
18. The Last Mile: State, Audio, and Frozen Dependencies 42

Part VI: What v1.0.2 Means

19. A Release Is a Set of Verified Boundaries 45
20. The Numbers, and What They Do Not Mean 47
21. The Deliberate Unfinished 49
22. Rules Learned the Expensive Way 52

Epilogue: The Lens Does Not Replace the Page 55
Appendix A: A Condensed Development Timeline 56
Appendix B: The Architecture at v1.0.2 58
Appendix C: Selected Engineering Measurements 60
Appendix D: An Error Catalogue from the Road to Release 62
Appendix E: Glossary 64
About the Author: The Omnitome Engineering Team 66

Prologue: The Four-Thousand-Chapter Problem

The first design document was not a design document. It was a familiar irritation.

Imagine a reader following a sprawling serial novel. The world is enormous, the characters are numerous, and the story has enough history to sustain a small civilization. A single work may run to three thousand, four thousand, perhaps five thousand chapters. There are battles worth remembering, reveals that reorganize everything that came before them, funny exchanges that give a cast its pulse, and quiet turns in a character's life that make later choices matter. There are also chapters that walk in circles. A character reconsiders the same insult for the sixth time. A conversation repeats information the reader already knows. A scene has three extra layers of internal monologue before it reaches the one new fact that changes the situation.

The problem is not that the reader dislikes the story. The reader likes it enough to endure the drag. That is precisely what makes the friction interesting.

A conventional summarizer offers an unhelpful bargain: hand over the experience and receive the gist. It may tell us that a hero trained, fought a rival, and unlocked a new ability. It may leave out the tactic that made the fight intelligible, the small line that revealed a character's insecurity, the limitation that will make the ability dangerous later, or the contradiction that makes the victory more than a number on a power scale. The output may be short. It may even be accurate in a narrow sense. But it can also flatten the very thing the reader came for.

The original motivation for Omnitome was closer to a question than a product category: could a person experience more of a very long story in less time without turning the story into a lifeless synopsis?

That question later grew beyond fiction. If a reader could move through a huge serial novel with greater information density, perhaps the same machinery could help a student traverse an immense textbook, a researcher move through a stack of papers, or a professional revisit a dense technical manual. The common problem was not the genre. It was the widening gap between the amount of source material available and the amount of attentive time a person could spend with it.

This did not mean every sentence was disposable or that the machine could reliably recognize every moment a human reader would treasure. Those were exactly the hard parts. What was the system allowed to remove? How could it tell a recurring motif from a redundant paragraph? How could it retain causal steps rather than only conclusions? How could it preserve the original so the reader could challenge the compressed version? How could it tell the reader when it had failed to do better than a fallback?

The origin story preserved in the project conversations begins in late 2024 with local GPT4All summarizer experiments. The surviving materials do not provide enough detail to reconstruct every early prompt, model file, or benchmark. They do preserve the important direction: the project began as an attempt to put a language model to work on a real, personal reading problem, with local execution as an attractive part of the experiment. The version of Omnitome described here is not the inevitable result of that first attempt. It is the outcome of many corrections to the original premise.

Over time, one phrase became the most compact expression of those corrections: **compression, not summarization**.

Summarization asks, “What is the gist?” Compression asks, “What is the smallest useful representation that still carries the important structure?” The latter is a harder requirement. It requires a model, but it also requires document parsing, semantic retrieval, chapter planning, word-budget enforcement, attribution, user interface design, database lifecycle rules, testing, and a willingness to show the source again when the compressed lens is not enough.

The path from a local summarizer to a packaged desktop application was not a straight line. It passed through an older product name, a rebuilt frontend, an embedding migration, a growing parser, a GPU that could run out of memory in unexpected places, a text-to-speech engine that worked until it was frozen into an executable, a Windows installer that met a two-gigabyte tool limit, and audio playback that was technically successful but not audible to the user.

This is the story of those turns. It is not a tale of AI replacing reading. It is a record of trying to make reading more navigable, more personal, and more accountable, one difficult engineering boundary at a time.

PART I: THE FRUSTRATION BEFORE THE ARCHITECTURE

1. The First Local Summarizer

The earliest remembered version of the project belongs to late 2024. Its tools were more modest than the application that eventually carried the Omnitome name. The idea was to experiment with GPT4All locally and ask a language model to summarize long reading material. That starting point matters for two reasons. First, it places the first experiment in the lineage of local inference rather than in a later decision to add a privacy feature to an already cloud-dependent product. Second, it shows how little of the final product was present at the beginning.

A summarizer can be deceptively easy to prototype. Feed it a passage, ask for a shorter version, and judge the result by whether it reads well. The first successful output gives a gratifying signal: the model has understood something, and it can express that understanding more compactly. The danger is that a readable paragraph can conceal several different failures. It may omit a necessary premise. It may merge two similar but distinct events. It may invent a transition. It may preserve the ending of an argument while losing the qualifications that keep the ending from being misleading. In fiction, it may retain the plot point while deleting the beat that makes a character's decision emotionally coherent.

The earliest GPT4All phase is not documented here as a sequence of reproducible experiments. The available source material does not establish a definitive model version, quantization format, context size, hardware configuration, or measured compression ratio for those tests. It would be easy to invent such particulars, but doing so would turn a history of engineering into a story about engineering-shaped props. The honest account is simpler: local LLM summarization was the first exploratory mechanism, and the product matured as the team became more precise about what the output needed to preserve.

Even that modest beginning contained a durable tension. A local model promised control over where the computation happened. It could reduce dependence on a remote service, limit routine content transfer, and let the software remain useful without an internet connection, provided the required weights and dependencies were available. But local inference was not free of constraints. Models occupied memory, ran at different speeds, and produced different quality levels. Running a model on one's own machine transferred the responsibility for performance from a service provider's infrastructure to a consumer system with finite RAM, finite VRAM, and other applications competing for the same resources.

This trade-off would become a recurring character in the story. "Local" would not remain a slogan floating above the architecture. It would become a requirement that had to survive model downloads, first-run onboarding, offline voice loading, database boundaries, cold starts, and actual packaged execution.

The early summarizer also left the central editorial question unresolved. A short output is not automatically a better reading experience. A summary may be efficient for remembering a conclusion, but it is not necessarily suitable for experiencing a narrative or understanding a

complex system. To work for the original motivation, the application had to identify a narrower target: remove repeated material while retaining the information that makes later material meaningful.

That target is neither purely linguistic nor purely technical. It resembles an editorial judgment. One needs to recognize the structure of the source before compressing it, and one needs a way to inspect the result rather than accept the model's confidence as proof. Those requirements led away from the idea of a single all-purpose prompt and toward the system that Omnitome eventually became: a sequence of explicit transformations with source material preserved as a separate representation.

In retrospect, the first local model was not the product. It was the question in executable form.

Could computation help someone move faster through a mountain of text? Yes, sometimes. But could it do so without silently replacing the source with a plausible account of the source? That became the real project.

The prototype's first inheritance

The first inheritance was local inference. The second was skepticism about the word *summary*. The third was personal usefulness as the measure of success. The software did not start with a market map or a general claim that all reading should be automated. It started with a reader who had material too long to move through at the desired pace.

This origin explains why Omnitome's later work on provenance and loss-bounded compression is not decorative overengineering. Those features respond to the most dangerous failure mode of the original idea: the machine might make a book shorter by making it less true to the reading experience.

A summary can be judged by whether it contains the major points. A reading lens must also be judged by what it allows a reader to experience, question, return to, and remember. That difference would ultimately determine the architecture.

2. The Product Hiding Inside the Prototype

The long-novel problem appears at first to be a content problem. Too many chapters, too much repetition, too little time. Yet the moment software attempts to solve it, the problem spreads into several disciplines.

The input has to be parsed. The system must determine where chapters and sections begin, distinguish headings from ordinary text, avoid mistaking a table of contents for the real narrative, and handle different encodings or document formats. It then has to organize the text into units that are large enough to preserve context but small enough to fit into an embedding model and a generation prompt. It needs to find relevant source passages for the chapter currently being synthesized. It needs to decide what to ask the language model to do. It needs to verify that the output falls within a useful length range. It needs to keep a relationship between each generated claim and the material that supports it. Finally, the application needs to present the result in a way that lets a human read it without feeling that the machinery is constantly interrupting.

The early product framing could reasonably be called “an AI reader.” That label was convenient but too broad. Many reading applications can put text on a screen, and many AI tools can produce summaries. Neither description captures the real difficulty. The project was not merely inserting a chatbot next to a document. It was designing a pipeline that transforms one representation of a work into another while keeping enough of the original available to audit the transformation.

The story of a long serial also imposed a special obligation. The system had to preserve more than facts detached from their order. Plot depends on sequence. Character development depends on changes in belief, motive, and relationship. A reveal often changes the meaning of a paragraph that appeared harmless many chapters earlier. In technical material, the equivalent structure is a chain of definitions, assumptions, and conclusions. A compression system that keeps the conclusion but discards the premise may feel efficient until the reader tries to apply the idea.

The project therefore needed to respect the source’s internal organization before asking a model to rewrite it. Chapter titles, section boundaries, paragraph boundaries, citations, and later document metadata were not peripheral conveniences. They were scaffolding for preserving meaning.

The second requirement was epistemic humility. A system cannot guarantee that every generated sentence faithfully reflects its source simply because a model was prompted to be accurate. Attribution can show where a sentence was grounded, but the existence of a link does not prove that the sentence is a perfect paraphrase of the linked passage. Retrieval can identify candidate evidence, but similarity is not the same as logical entailment. The human reader remains the final judge. A credible system should make that judgment easier rather than attempt to make it unnecessary.

From this perspective, the product’s later three laws were not merely aesthetic language. “Source is paper” meant that the source view must be calm, neutral, and authoritative. “The lens is tinted” meant that the derived reading could carry its own visual identity. “Derivation never masquerades as source” meant that the application must make the difference legible and preserve the route between the two. These ideas eventually became part of the named Two-Representation Invariant.

The product also needed to work as a library rather than as a disposable summarization session. An imported work might have thousands of paragraphs, embedded metadata, OCR-derived text, classification tags, summaries, synthesized chapters, and a local semantic index. A source could be reprocessed. An Omni reading could be removed while the research source remained. A large original file might be detached to save space while the smaller derivative remained useful. The application had to know which material existed, what had been generated, and which state each object occupied.

That means the real object being built was a local knowledge workflow. The reading screen was its most visible surface, but the workflow behind that screen was where much of the engineering risk lived.

A useful ambition, made narrower

There was also a temptation to build the entire imagined future at once. If a text could be compressed into a reading view, perhaps it could become audio, a discussion, illustrated scenes, or eventually a personalized visual adaptation. The project conversations preserved that broader direction: reading, audio, and future story experiences were related possibilities rather than unrelated feature requests.

The engineering decision, however, was to build the part that could establish the foundation. That meant concentrating on ingestion, local semantic understanding, synthesis, provenance, and a reliable reading interface. Story and discussion ideas could remain in the long horizon without being smuggled into the v1.0 release as unfinished obligations.

This is one of the first lessons of Omnitome: the size of a vision does not justify the size of every immediate task. A broad vision is useful when it helps choose the next foundation. It becomes harmful when it insists that every branch grow before the trunk is strong enough to carry it.

3. Smart Reader Learns What It Is

The first public-facing identity was **Smart Reader**. In parts of the early project, the software was described in language associated with AI-powered manga or story reading. That described the motivating use case but could not describe the mature product. As the ingestion and synthesis pipeline widened, the software could accept novels, textbooks, research papers, articles, Markdown notes, and other documents. The reading problem had become larger than the original format.

The evolution did not happen in a single rebrand. It happened through an accumulating set of architectural decisions. The project began to distinguish source extraction from semantic indexing, classification from summarization, editorial planning from generation, and provenance from the text that provenance supported. Each separation made a hidden assumption visible.

The surviving repository-era logs show a staged program rather than a single rewrite. Early phases worked on document structure and progressively synthesized chapters. Other phases repaired source slicing, bounded the compressor, improved chapter title resolution, added reader-facing source navigation, and then widened the import workflow. Phase labels such as 4.7, 4.26, 5.1, 5.2, and later 5.8 or 5.9 were internal engineering markers. They record the fact that the product's surface was being built on top of an expanding set of tests and contracts.

A version or phase number is not a quality guarantee, of course. The logs repeatedly show why. A capability could be marked complete while an edge case remained undiscovered. A route could render without a type error and still misresolve a chapter. A user interface could display progress without reflecting the actual backend job. A fallback could keep the screen from going blank while masking a provider failure. Closing a phase therefore required more than implementing its headline feature. It required reviewing the boundary where that feature met the rest of the system.

The project increasingly adopted a working rule: reconstruct the current state before changing it. When a session ended or a context limit interrupted the work, the next session was expected to read the handoff, inspect the repository, compare current files with prior records, and verify the exact commit before taking action. The saved logs preserve repeated warnings about stale editor buffers, accidental changes, incorrect assumptions from earlier summaries, generated-file corruption, and risky Git recovery.

These are not glamorous parts of software engineering, but they form an important part of the project's identity. An AI-assisted development workflow can move quickly, yet speed is valuable only if the system remains legible. If an agent says a feature is complete but the code or test output disagrees, the agent's prose is not the source of truth. The source of truth is the actual repository and its behavior.

That principle echoed Omnitome's own user-facing promise. The derived reading should not masquerade as original source; the engineering handoff should not masquerade as verified state.

A project with an increasing number of obligations

As Smart Reader evolved, each new capability introduced a new obligation. Semantic embeddings introduced model loading, vector dimensions, pooling behavior, and GPU memory. A richer frontend introduced route state, asynchronous API coordination, and runtime errors that type checking could miss. Progressive synthesis introduced retry semantics and partial states. Provenance introduced a contract between generated text, source chunks, and the Research interface. Multiple import sources introduced a queue that had to normalize different inputs into one lifecycle.

The product became more capable, but it also became less forgiving of vague architecture. Once several subsystems depended on each other, a local fix could break a distant assumption. A reranker timeout might create a weak source match. A weak source match might undermine a compressed sentence. A frontend cache delay might make a successfully imported book appear missing. A startup shortcut might accidentally launch models before the machine had enough memory. These were not separate annoyances. They were signals that the product needed explicit boundaries and explicit state.

Smart Reader was becoming a reading system. The name had not yet caught up with the engineering.

PART II: TURNING A READING IDEA INTO A SYSTEM

4. Structure Before Intelligence

Language models attract attention because their outputs are easy to see. Document structure is quieter. It is also where a large part of the final quality is decided.

Suppose a PDF contains a table of contents, a preface, a sequence of chapters, footnotes, and appendices. A parser that flattens everything into a long string may leave the words intact while destroying the structure needed to interpret them. A section heading may be mistaken for a paragraph. The table of contents may be processed as though it were the book's real content. A footnote may be detached from the point it qualifies. A scanned page may yield no text at all. A Japanese novel encoded in Shift-JIS may arrive as unreadable characters if the byte stream is treated as UTF-8. The language model cannot repair every loss introduced before it receives the input.

The engineering record shows the project progressively building a structure-first pipeline. Depending on format, parsing involved tools such as PyMuPDF for PDF text extraction, EPUB spine and XML/XHTML parsing, `python-docx` for DOCX, RTF text extraction, CommonMark-aware Markdown parsing, web article extraction, and OCR for scanned pages. These were not all interchangeable routes. They needed format-aware handling, common downstream contracts, and tests demonstrating what survived extraction.

The pipeline gradually became explicit about its stages. In the more mature architecture, an import could proceed through ingestion and parsing, semantic indexing, classification, editorial outline planning, synopsis and synthesis, then attribution and final lifecycle reconciliation. Earlier logs describe related five-stage flows; later documentation names the fuller sequence with attribution as a distinct stage. The exact number of displayed stages evolved as the implementation became more observable, but the principle stayed the same: each stage performs a defined job, and the system should know whether that job actually completed.

The first stage establishes text that the application can work with. The second creates semantic chunks and embeddings. Classification determines the kind of work and labels it; it should not be confused with rewriting the work. Editorial planning tries to recover an intelligible chapter or section structure. Synthesis then generates the derived reading units, with retrieval and reranking supplying source context. Attribution maps those derived claims back to source passages. A queue and lifecycle layer coordinates all of it.

A key decision was to keep the application database under one writer. The Node.js backend owns orchestration and SQLite writes. The Python sidecar executes compute-heavy tasks such as embeddings, reranking, OCR, parsing support, and speech synthesis, but it should not independently write the application's SQLite database. This boundary reduces the chance of competing writers creating inconsistent lifecycle state or database contention. The frontend displays state; it does not become the authority that decides whether a job exists or is complete.

It sounds obvious after the architecture is drawn. It was not obvious enough to avoid repeated bugs.

The lesson of the misleadingly simple pipeline

A pipeline can be drawn as five boxes and still fail in many ways. One early diagnostic cluster found fields crossing boundaries under inconsistent names, such as `book_id` and `bookId`. Another found synopsis generation happening twice. A stage could execute without a durable job record. The execution order could differ from the planned dependency order. A task could complete on the server while the user interface kept an old cache. A provisional database row could remain visible after an import failed.

These are ordinary integration defects, but they strike at the user's mental model. If the UI says "finished" when the derivative does not exist, or suggests a document has been deleted when the database merely has stale state, users cannot tell what happened. A system that transforms personal documents must be able to account for its actions.

The right response was not to add more celebratory progress animation. It was to make each stage's work observable, preserve the queue state, distinguish failure from waiting, and reconcile the final lifecycle state once the synthesized chapters actually existed.

The larger implication is simple: intelligence does not begin at the model boundary. It begins with a faithful representation of the input. The better the structure supplied to the model, the more meaningful the generation can be. The less ambiguous the pipeline state, the more trustworthy the final interface becomes.

5. September: Rebuilding the Semantic Foundation

By September 17, 2026, the project was far enough along to expose a frustrating gap between apparent speed and actual semantic quality. The system had a local embedding component described as a 256-dimensional `LocalDenseSemanticVectorizer`. Some early checks were fast, but the session records question whether that component provided the depth of semantic understanding required by the product. The documentation itself was cautious: the speed and observed similarity scores suggested some useful signal, but the implementation might have been using a hash-like technique or a small projection rather than a full neural embedding model. That was a diagnosis to investigate, not a claim to repeat without evidence.

The chosen next step was to replace the older vectorizer with **BGE-M3**, a multilingual embedding model that produces 1,024-dimensional dense vectors. It was a substantive architectural pivot. The system wanted to work across more than one language, and the embeddings had to be capable of connecting passages by meaning rather than merely matching visible vocabulary. A chapter on machine learning might express an idea differently from a passage in a research paper, but the retriever still needed to find the relevant evidence.

The migration came with a useful reminder: choosing a model is not the same as integrating a model. Pooling strategy matters. The first pooling implementation used mean pooling, but the recorded comparison found better results with CLS pooling for this implementation. The team ran targeted checks and made pooling a concrete architecture decision rather than treating it as a detail that could remain arbitrary.

This period also clarified the hybrid arrangement. Semantic vectorization stayed local. Retrieval stayed local. At that point, text generation and some classification paths still used OpenRouter with DeepSeek V4 Flash. The system could keep document parsing, embeddings, and retrieval on the user's machine while sending the selected synthesis work to a configured cloud provider. This was more precise than calling the entire product fully offline. The project was local-first, but a cloud generation provider meant content could leave the machine when that route was used.

That distinction became increasingly important as the local LLM option matured. A real local-first product needs a clear statement of which operations are local, which may use a network service, and what the user has to configure. It should not turn "local embeddings" into an unqualified "nothing ever leaves your machine" promise if a cloud model still receives prompts.

Retrieval is a chain, not a switch

The BGE-M3 migration did not by itself guarantee good attribution. Embeddings created candidate source matches, but the candidates still needed to be ranked in context. The project brought in **BGE Reranker v2** as a cross-encoder step: rather than relying only on embedding similarity, the reranker evaluates query-passage relevance more directly. Later logs show input-calibrated timeouts and batching work, because a large chapter should not be treated like a tiny paragraph with the same fixed deadline.

The chain therefore became: parse correctly, chunk sensibly, embed the chunks, retrieve candidates, rerank them, then provide relevant context to synthesis and preserve the chosen source link. A flaw in any link can weaken the next. A semantic index cannot recover text the parser lost. A reranker cannot invent support if the correct passage was not indexed. A fluent generation cannot guarantee that its attribution is exact. Good engineering requires the team to test the whole path and inspect its intermediate outputs.

The September sessions also developed a more disciplined workflow around benchmark evidence. Rather than infer that the new model was better because it sounded advanced, the team tested the implementation, reviewed actual outputs, and compared alternatives. The transition from 256 dimensions to 1,024 was not useful merely because the number was larger. It was useful because the product required multilingual semantic retrieval and the new model offered a better foundation for it.

By the end of that work, BGE-M3 had become the local semantic core of Smart Reader. That decision would later collide with the 4 GB GPU constraint, force FP16 optimization, and shape resource lifecycle management in the desktop version. A good architectural choice can solve one problem while introducing a new one. That is not a contradiction. It is the normal cost of making a system more capable.

6. React, Routes, and the Shape of the Reader

The semantic refactor was only half of the September rebuilding effort. The project also moved from a legacy vanilla HTML, CSS, and JavaScript interface toward a React application built with Vite, TypeScript, and Tailwind CSS. The order mattered. The roadmap put BGE-M3 first and the React migration second, because the user experience needed to be designed around the product's real data and provenance contract, not around a superficial mockup.

The React work established the Library route, book cards, search and filters, Book Details, the reader surface, and routing between the different views. It turned an interface that had evolved with the prototype into a more structured application. The result was not simply a change of framework. It gave the codebase clearer places for data fetching, route state, reusable components, and interaction logic.

Different screens served different purposes. The Library was a catalog of derived reading experiences. Book Details offered the chapter list, synopsis and metadata, and operations such as opening or synthesizing content. Research became the source-facing workspace. Import became an entry point for files, web material, and pasted text. The reader itself could display the compressed chapters and provide a route back to supporting source passages.

That separation made the Two-Representation Invariant visible in the interface. It was no longer enough to store an original string and a generated string somewhere in the backend. The application needed distinct navigational surfaces with clear meaning. The source belonged in a research-oriented view. The derived reading belonged in the Library and reader workflows. If a citation was clicked, the reader should arrive at the corresponding source location instead of being shown a vague citation label with no useful context.

The migration was not entirely smooth. Session logs note a failure caught by the Vite/Babel runtime even though TypeScript's checker had not surfaced the same problem. That incident is worth preserving because it corrects a common misconception: static typing is one form of validation, not a universal proof of runtime correctness. A successful typecheck can tell a developer that certain relationships are well-formed under the type system. It cannot certify component behavior, module transformation, network state, or every browser/runtime interaction.

The project also had to make typography and reading behavior canonical. A technical tool can be accurate and still feel exhausting if headings jump unpredictably, source text and generated text are visually indistinguishable, or a long reader pane behaves unlike a book. Later refinements included chapter filtering by title or index, a full-screen toggle with F11 support, separate scrolling for Research navigation and content, and theme-aware scrollbars. Those are not core semantic features, but they improve the conditions under which the semantic features can be used.

The frontend does not own the truth

The React work reinforced an architectural rule that became increasingly important: the UI should display processing truth rather than fabricate it. It can provide immediate feedback when a file is accepted. It can show a progress bar and a status message. But the backend queue is the durable record of what is running, what has completed, and what failed. The UI must reconcile its cache with that record after a task completes.

A later bug exposed the difference. An imported book had completed its pipeline, but the Library catalog failed to show it because a lifecycle row had been initialized before smart chapters were committed. The frontend could refetch, but refetching alone could not fix an incorrect lifecycle flag. The backend needed reconciliation hooks at synthesis completion, queue completion, and lifecycle access, plus a startup backfill for older rows. The application then added event-driven invalidation so that a completed import could refresh the Library immediately.

This is a useful example of how frontend correctness depends on backend invariants. A beautiful route cannot compensate for inconsistent persisted state. React improved the ability to structure the experience, but the product still depended on a reliable system underneath it.

The lesson was not “use React.” It was that a long-running, asynchronous product needs a clear separation between presentation, state ownership, and compute. Frameworks help express boundaries. They do not create them automatically.

7. Compression Is a Contract, Not a Prompt

“Make this shorter while preserving the important points” sounds clear until a system has to decide what counts as important. A language model may produce an eloquent summary that is too short to retain essential distinctions. It may also produce a verbose response that restates the source with cosmetic changes. In neither case does the user’s actual goal get met.

The project addressed this by treating compression as an explicit contract. The source was divided into editorial units, typically around 1,500 to 2,500 words in the mature pipeline documentation, while the synthesized chapter target evolved toward a consistent range around 250 to 360 words, with a hard ceiling used in later contracts. Earlier records include different floor and ceiling rules, so those values should be read as a sequence of revisions rather than one unchanging specification. The important evolution was from open-ended prompting to adaptive, measurable output boundaries.

The first versions of this requirement were too crude. A simple target ratio could reject a useful passage or accept an output that met the length target while losing too much. A prompt could describe compression in strong language, but the model might still produce a long summary. A validator could reject that response, but repeated retries against the same unsuitable contract would not magically solve the reasoning problem.

The session record from September 20 describes an especially important intervention: the model task was reframed from “write a summary” into a more explicit compression assignment, with the system checking the output against the intended length behavior. The early experience showed that product requirements should exist in code as well as prose. If a chapter must land in a defined range, the pipeline should validate that range. If a source unit contains non-chapter material such as a contents list, the planner should not blindly promote it into a reader-facing chapter. If the first output fails, the system should distinguish a retry from a successful synthesis.

A later investigation also exposed a gap between marketing shorthand and measured performance. The repository’s headline described approximately 7:1 compression. A historical review of three cited examples recorded ratios of 3.61:1, 4.09:1, and 5.05:1, with a mean of about 4.25:1. The book must preserve that distinction. The 7:1 figure expresses the product’s intended or documented target framing in some materials; the three-case mean describes a limited historical measurement. Neither should be silently substituted for the other, and three examples do not define the distribution for all books. A responsible release needs a stable canonical corpus, a clear calculation method, and performance claims that can be reproduced against that corpus.

Why a ratio is not enough

Compression ratio is a useful number because it expresses the amount of material removed relative to the source. Yet two chapters with the same ratio can have very different quality. One may delete redundant transitions while preserving every major causal step. Another may retain all the names but lose the relationship between them. A ratio can flag a result that is much too long or suspiciously short; it cannot independently measure faithfulness, narrative continuity, or factual completeness.

This became part of the evaluation design. The project introduced tests and a corpus gate to inspect several dimensions rather than treating a word count as the only quality signal. Class A assertions represented universal invariants, Class B allowed tolerances where variation was expected, and Class C locked known regressions. The 10-fixture release gate and larger corpus benchmark were intended to catch different classes of errors. A clean parse did not necessarily mean a good compression. A good-looking compression did not necessarily mean the source attribution was sound.

A later feature made re-synthesis explicit for chapters that had timed out or fallen back to heuristic compression. The ability to run the pipeline again was a product decision as much as a recovery tool. A fallback might be preferable to a blank screen, but the application should not quietly present a degraded result as though it were equivalent to the full process. It should retain enough state for the user to understand what happened and, where possible, ask for a fresh synthesis.

In a reliable compressor, the prompt is only one part of the contract. The contract also includes how source units are created, what context is retrieved, how length is measured, what counts as a retry, how failures are labelled, how sources are attributed, and which regression fixtures must pass. The model produces content. The system creates the conditions under which that content becomes a product.

8. Two Representations, One Trust Boundary

The most consequential product decision was not a model choice. It was a separation.

The project came to call it the **Two-Representation Invariant**. Original Reading is the source, preserved without rewriting. Omni Reading is a derived lens, synthesized for higher information density. They are related, but they are not interchangeable. The original tells the reader what the source actually contained. The Omni version tells the reader what the system has constructed as a more compact route through that source.

That distinction is easy to state and difficult to enforce. A summarizer normally treats the input as temporary material that can be replaced by its output. Omnitome needed the opposite relationship. The source had to remain available after synthesis, and the generated text had to carry its derivation openly. The interface eventually reflected a concise maxim: **"Omni is the product; Original is the proof."** Earlier language sometimes framed Smart Reading and Original Reading as peers. That was later refined. Smart or Omni Reading became the primary product experience; Original Reading supplied the authoritative evidence and the means to challenge it.

The distinction affected both storage and navigation. The Research view represented the original document or a research dossier. The Library represented the active compressed reading. Each generated sentence was intended to retain deterministic provenance back to relevant source paragraphs or chunks. A user could follow a citation from the Omni side into Research, where the corresponding source passage was highlighted. In the other direction, the app's internal source and chapter identities helped keep the generated chapter connected to the work from which it had been derived.

The early provenance work was not a magical switch that made every output true. It took several phases. Phase 5.1 built the backend foundation for bibliographic metadata and source resolution. The work had to arbitrate among candidate source chunks, keep editorial and source namespaces from being confused, handle one generated segment that could map to more than one source block, and make the mapping usable from the frontend. Later phases added segment chips, anchored previews, highlighting, source scrolling, and more consistent behavior across long passages.

At one stage, the team had to decide how much detail a citation should expose in the interface. Too little granularity would make the provenance feel ceremonial: a link labeled "source" might technically go somewhere without helping the reader verify anything. Too much detail could turn the reading surface into a dashboard of identifiers. The design compromise was to make the interaction approachable, using compact sentence-level or segment-level markers while keeping the resolution contract anchored in source paragraphs, pages, and chunk ranges. The interface could be quiet; the underlying provenance could still be precise.

Citation is a relationship, not a decoration

A citation has at least three responsibilities. It should identify a source location. It should allow the reader to reach that location. And it should not claim more than the underlying mapping proves.

The first two can be implemented through data structures and navigation. The third is a matter of epistemic honesty. If a synthesized sentence links to a source paragraph, the link establishes a trail of origin. It does not automatically prove that the sentence is a logically valid inference from that paragraph. A sentence may combine evidence from several chunks. A retrieved passage may be relevant but incomplete. A model may overstate a conditional claim. A citation system can make such errors easier to uncover, but cannot guarantee they never occur.

This is why “trustworthy AI” in Omnitome is a design direction, not a magical property. The system makes its transformations visible. It preserves the source. It gives readers a path to check claims. It uses tests to protect the mapping. It makes degraded synthesis states recoverable. Trust becomes a practice of inspection, not a request that the user believe the model.

Lifecycle states protect meaning

The same idea later extended to lifecycle management. A source could exist in Research without having an active Omni reading. A library entry could remain after a heavy source file had been detached. An item could be archived rather than actively displayed. These states should never be reduced to a single ambiguous “exists” flag.

The repository’s lifecycle model came to represent states such as active, research-only, library-only, and archived. The corresponding operations mattered. Removing a reading from the Library should not silently erase its Research source. Detaching a large source file should not pretend that the remaining compressed text was still fully backed by an accessible original. Permanently purging material should be a distinct, deliberate operation. In the user-facing application, the source-detached status became an explicit badge rather than an invisible data loss.

One October bug showed why this lifecycle could not be left to happenstance. An import could finish synthesis after a usage record created a lifecycle row with `in_library = 0`, because no smart chapters existed at the exact moment the row was first created. Once chapters appeared, nothing automatically corrected the state. The book had completed work but was absent from the Library. Fixes were added at several points: the post-ingestion hook, the queue completion hook, the `ensureLifecycle` service, and a startup database backfill. The redundancy was intentional. It gave both the normal path and older records a chance to converge on the correct state.

That bug belongs in the chapter on provenance because source trust is more than a footnote marker. It includes being able to account for where the original lives, whether the derived reading exists, and whether the application is accurately describing that relationship.

9. Import Became a Product of Its Own

The import screen began as a door into the pipeline. It eventually became a small product with its own routes, queue, feedback, progress model, retry logic, duplicate handling, and recovery behavior.

Three input paths crystallized into the core experience: **File Upload**, **Web Discovery**, and **Paste Text**, alongside a dedicated **Import Queue** view. That design mattered because users do not collect knowledge in one file format. They might have an EPUB from a book source, a PDF textbook, a copied research excerpt, a long web article, or a Markdown transcript. The application should normalize those inputs into a shared workflow rather than build a separate miniature product for each one.

The early import system had false confidence in its progress display. A visual bar could advance because the UI had a list of stages, even if the underlying task state was not being reported reliably. The team replaced this fiction with observable jobs. Queue tasks acquired persistent status, stage, progress, and retry behavior. The backend could distinguish a file accepted for processing from a work that had completed synthesis. The frontend could show which task was active and make cancellation or retry available at the appropriate stage.

The workflow also exposed several integration mistakes. Some code used `book_id` where another layer expected `bookId`. Synopsis generation could be duplicated. Synthesis lacked job logging in an earlier path. The execution order did not match the dependency order. Paste import did not have a full ingestion route. Web import handled multi-source work synchronously rather than through the same observable job architecture used elsewhere. Those problems were fixed by aligning the paths around one queue and one pipeline rather than adding isolated patches to each input method.

Progress reporting became more deliberate as the system matured. The later normalized stage map assigned monotonically increasing weights, with parsing at 15 percent, semantic indexing at 30, classification at 45, outline generation at 55, synopsis at 60, synthesis at 70, attribution at 85, and completion at 100. These percentages did not represent a universal estimate of remaining seconds. They represented progress through a staged workflow and prevented the bar from appearing stuck during semantic indexing. The distinction matters: a progress bar is a communication contract, not a stopwatch.

The background queue changed the reading rhythm

A user should not have to sit in front of an import screen for the entire duration of a long book’s processing. The queue let the task continue in the background and allowed the user to navigate elsewhere. The interface eventually included a persistent import dock and a cinematic banner for the active task. The visuals could acknowledge that substantial work was happening, while the queue remained the durable record of the job.

The queue also changed what “failure” meant. If an import failed at the beginning, the application needed to remove or mark the provisional book row rather than leave a ghost that looked ready. If a source parsed but synthesis timed out, that was different from a corrupt file. If all work had

completed but the Library cache was stale, the remedy was a refetch, not a second import. This required consistent status transitions, startup cleanup for abandoned `processing` rows, and careful synchronization between the database and UI query cache.

A more complete import experience also needed duplicate protection. In the v1.0.2 polish cycle, incoming file buffers and pasted text were hashed with SHA-256. If identical content already existed, the application presented a replace-or-cancel decision rather than allowing accidental duplication to quietly fill the Library. The hash did not solve all possible semantic duplicate cases; it addressed identical content bytes or text under the implementation's chosen normalization path. That is still a useful, bounded guarantee.

The queue's self-healing behavior expanded later. Completed tasks could emit an `omnitome:books-changed` event, and the Library subscribed to it. Query data was refreshed after completion and when a user opened a recently completed import. The point was not to force users to learn cache invalidation. It was to ensure that the application behaved as though a completed task had actually happened.

Importing is the first trust test

An import is often the user's first interaction with a product. If it produces garbled text, a false chapter list, a misleading synopsis, or an unexplained failure, later strengths do not matter. The user has already learned that the application is difficult to trust.

For Omnitome, ingestion also determines the quality ceiling for all later stages. Incorrect chapter segmentation affects editorial planning. Bad character decoding poisons embeddings. A table of contents mistaken for content can become a synthesized chapter. A missing scanned page cannot be recovered by a clever prompt. Queue honesty, parser quality, and downstream grounding therefore belong in the same reliability story.

The growing import subsystem embodied a practical principle: a long-running AI application is not one model call. It is a workflow that must be visible, resumable where possible, and honest about what finished.

PART III: REAL DOCUMENTS, REAL LIMITS

10. Documents Refuse to Behave

Once the pipeline was exercised against real documents, the tidy abstraction of "a document is text" began to dissolve.

PDFs are not always books. They may be text pages laid out in columns, scanned page images, annual reports with tables, mathematical material with unusual glyph encodings, or a mixture of text and figures. EPUB is not simply a text file with a different extension; it is a package of structured XHTML documents linked through a spine and sometimes accompanied by navigation metadata that is useful for chapter detection. DOCX and RTF have their own structural and formatting conventions. Markdown can contain headings, fenced code, lists, tables, and chat transcripts whose turns carry meaning. Web pages contain navigation, advertising, menus, and other material that should rarely become part of the reading body.

The parser had to choose what to preserve, what to normalize, and when to invoke a heavier processing path. PDF text extraction used PyMuPDF. Scanned pages could trigger PaddleOCR when text extraction yielded no characters. EPUB ingestion followed XML/XHTML structure. DOCX used `python-docx`, while RTF used a dedicated text extraction dependency. Markdown required a structure-aware parser and special treatment for some dialogue-heavy transcripts. Web imports used a readability-style extraction path and metadata helpers for sources such as academic papers and public-domain books.

These routes gradually fed a shared downstream model, but the input-specific edge cases remained real. A parser could accidentally include the table of contents as the first chapter. Level-one heading detection could promote decorative text into the structure. A math extraction experiment could over-detect ordinary text as formulas. Some PDFs used legacy private-use-area code points where a character that visually looked like a mathematical symbol did not correspond to the expected Unicode value. The team added targeted normalizers and, in one case, gated the experimental math extraction feature off because its false positives made it too risky to leave enabled by default.

That last decision is important. Feature work is often portrayed as an accumulation: parse more, recognize more, add more intelligence. But engineering judgment sometimes means disabling an ambitious capability until its error modes are understood. A feature that confidently corrupts an input is worse than a narrower parser whose limitations are known.

Language is part of structure

The multilingual ambition pushed the parser beyond the English default. The Japanese Aozora and Shift-JIS work was a clear example. The pipeline added a fast UTF-8 path, statistical charset detection for legacy encodings, maximum chunk-size guards, bounded embeddings, CJK-aware word counting, and a heading normalizer for Aozora-style sections. Japanese prose cannot be reliably segmented by the same whitespace assumptions used for English. Treating the count of space-separated terms as a universal word count distorts chapter sizes, summaries, and compression ratios.

The broader pipeline also recorded handling for French and European UTF-8 text, Russian encodings such as CP1251 and KOI8-R, Chinese legacy encodings such as GB18030 and Big5, and other multilingual edge cases. The goal was not to claim perfect language support. It was to recognize that decoding and structure extraction must be tested across the actual language characteristics of the target inputs.

A particularly instructive failure happened after an Aozora heading rule was introduced. A change intended to recover chapters created a title artifact because the parser's hierarchy and the book's true structure were not as simple as the first fixture suggested. The subsequent fix was constrained to the relevant heading behavior and verified against real pipeline evidence. The correction also came with a caveat: a passing narrow test was not enough to establish universal chapter detection. The project had to re-import the source and inspect the resulting outline.

The corpus became a design tool

Testing eventually moved beyond one canonical PDF. The project developed a corpus of real and edge-case inputs, including novels, textbooks, research transcripts, multi-language public-domain works, scanned material, and documents that could be misclassified as something else. The v1.0 corpus gate covered ten fixtures across several formats, while the broader empirical document corpus recorded twenty-two files, close to a thousand extracted chapters and about 2.46 million words.

A corpus can expose assumptions that a unit test cannot. A test may prove that a parser function returns a string. A set of real documents can reveal that the string is in the wrong order, that headings were duplicated, or that the parser has quietly consumed a navigation section as prose. Test fixtures also need to be rights-aware. A convenient source document is not automatically lawful to redistribute inside a public repository or a commercial archive. That concern later led to a repository history cleanup and more deliberate licensing documentation.

Document diversity makes one thing clear: ingestion is not a prelude to intelligence. It is the first form of intelligence the system needs. Before Omnitome can compress a text responsibly, it has to know what text it has.

11. The Four-Gigabyte Ceiling

The project did not develop on an unlimited cloud GPU. Its benchmark machine was a consumer laptop: an AMD Ryzen 7 4800H, 16 GB of DDR4 memory, and an NVIDIA RTX 3050 Laptop GPU with 4 GB of VRAM. Those specifications belong in the history because they explain the shape of the architecture. The system had to perform neural retrieval, reranking, OCR when needed, and eventually voice synthesis without assuming that a dedicated workstation could hold every model in memory at once.

At first, BGE-M3's GPU footprint was large enough to make the constraint painfully visible. The benchmark documentation records active VRAM use around 3,959 MiB before optimization. Under that pressure, large documents and batching could push the machine into paging or memory failure. An operation that worked on a small sample might become dramatically slower on a large chapter because the GPU began moving data between VRAM and system RAM over PCIe. The application could be "running" while delivering such poor latency that the experience was effectively broken.

The response was not one magic optimization. It was a series of changes with distinct purposes.

First, the embedding model was loaded in FP16 when CUDA was available, reducing the model's active memory footprint. The recorded comparison brought BGE-M3's VRAM use down to around 1,135 MB. Second, embedding work was batched with bounded sequence lengths and controlled micro-batches instead of allowing giant inputs to consume unbounded attention memory. The configured maximum sequence length was 2,048 tokens in the relevant path, with batch sizes constrained for the hardware. Third, warmup and telemetry made model readiness and import progress more observable. A CUDA allocator setting using expandable segments was also recorded as part of the tuning work. These changes collectively made GPU inference more predictable.

The benchmarks report a sixteen-chunk embedding case falling from roughly 124 seconds to under one second after the combined batching and FP16 improvements. That number is striking, but it should be read as a measurement from a specific historical comparison, not a guarantee for every sixteen chunks or every machine. The old path and optimized path differed in more than one dimension, and workload composition matters. The sound takeaway is that bounded neural work can be dramatically faster than a path that triggers memory paging.

Another problem existed outside the neural model. The Node.js ingestion pipeline allowed too much asynchronous work to accumulate. In the recorded benchmark, peak Node memory reached approximately 14.6 GB under the earlier behavior, which was unacceptable on a 16 GB system. The F65 sequential await barrier introduced backpressure by making dependent work wait for earlier work rather than launching an unbounded swarm of tasks. The later figure was around 296 MB of peak Node memory for the measured workload, described in the benchmark as a roughly 98 percent drop.

That is a different kind of optimization from reducing GPU memory. FP16 made a model's representation more compact on the GPU. The await barrier controlled how much application work was in flight. Both were necessary because the application had to fit within the machine as a whole.

The machine is a resource budget, not a footnote

A desktop application should not load every expensive model as soon as its window opens. On a 4 GB GPU, keeping BGE-M3, a reranker, OCR, and a speech model resident together could create unnecessary competition. Later work introduced an idle lifecycle manager that could evict model weights when they were not needed. The benchmark records described about 3.2 GB of VRAM returned to the operating system when models were evicted, with idle usage around 174 MB. A reloaded model had a cost, but on-demand loading made the whole application more livable.

This created a set of trade-offs. Warming BGE-M3 on startup could improve readiness for the most central reading pipeline, but warming every model could delay boot. Keeping TTS loaded could reduce the pause before speech, but it would occupy memory while the user was only reading silently. A large document could require a longer timeout than a small one, so fixed request deadlines could mislabel slow-but-valid work as failure. The system eventually introduced dynamic reranker timeouts scaled to the input, with values in the documented path ranging from approximately eight to sixty seconds.

The lesson was not to tune every operation to its best-case latency. It was to design the product around real resource limits, including cold starts, contention, and the cost of keeping a model alive.

Backpressure is part of the user experience

Backpressure may sound like an internal concern, but the user feels it directly. Excessive concurrency can make the interface lag, cause the machine to swap, and make every operation slower. A bounded pipeline might take slightly longer in an artificial microbenchmark while being far more reliable on actual hardware. A model that loads on demand may have a noticeable first-use delay but allow the user to continue using other parts of the application without memory starvation. Honest telemetry makes those trade-offs understandable.

The 4 GB ceiling became one of Omnitome’s most important teachers. It forced the team to stop treating compute as an unlimited service behind an API and begin treating it as a lifecycle-managed resource. That principle would carry into the Tauri resource manager, first-run setup, lazy-loading decisions, and the ability to operate with CPU fallback where supported.

12. Failure Must Be Visible

Reliability work rarely looks impressive in a promotional screenshot. It looks like a failed import marked as failed instead of left as “processing.” It looks like a timeout reported as a timeout instead of disguised as an ordinary synopsis. It looks like a re-synthesis action after a heuristic fallback. It looks like a progress bar that does not freeze at thirty-five percent while the backend is still working. Yet these details determine whether the user can tell what the system has actually done.

One recurring lesson in the session records was that AI output should be treated as an implementation input, not unquestionable truth. An agent might suggest a fix, describe a successful test, or infer that a file change has landed. The team had to verify the result at the current Git state, examine raw output, and run the relevant suite. The same skepticism was applied to the application’s own model outputs. A fluent synopsis was not sufficient evidence that the parser had found the real chapters. A fast embedding result was not sufficient evidence that the embedding model was neural. A successful backend request was not sufficient evidence that the frontend had displayed the audio.

The project accumulated a small but important operating code:

- Read the current repository before relying on a previous handoff.
- Make one workstream the focus of a session.
- Prefer raw terminal output, test logs, diffs, and actual application behavior over a generated status summary.
- Do not claim the full suite passed when only a targeted test ran.
- Do not commit from a dirty working tree or with red tests.
- After a change, inspect the exact `HEAD` and verify the behavior again.
- Stop repeated blind retries and move to diagnosis when the same failure recurs.

These rules emerged in response to real workflow hazards. A VS Code buffer could be stale relative to the file on disk. A Git extension or checkout could make the editor’s view disagree with the repository. Generated-file manipulation through PowerShell could change encodings or corrupt content. A destructive Git recovery could discard work if the actual state had not been established. Long-running agent loops could spend substantial time changing code without narrowing the problem.

This book does not present AI-assisted coding as either magic or menace. It can accelerate routine transformations, explain unfamiliar errors, compare designs, and produce useful patches. But it can also amplify a wrong assumption at machine speed. The discipline must therefore be evidence-oriented. The developer remains responsible for the state that ships.

The difference between green and trustworthy

A green test suite is a valuable signal, but a suite only verifies the behavior it encodes. Unit tests might pass while a frozen executable is missing a dynamic library. Cargo tests might pass while the child process leaks after the window closes. A backend endpoint can return a valid WAV while

WebView2 refuses to play it. A model may load in an existing virtual environment but attempt to download files when run with offline flags. A clean development launch tells us little about the behavior of a fresh installation unless the test actually uses a clean environment.

The project gradually separated several verification gates: backend unit and integration suites; Rust supervisor tests; TypeScript checks; Vite production builds; a multi-format corpus gate; direct test of the local sidecar; actual `tauri:dev` launches; and clean-install testing of the packaged Windows installer. These gates overlap, but none replaces all the others.

Failure visibility also applies to data lifecycle. If a source has been detached, the UI should label it. If a model is missing, the sidecar should return a controlled unavailable response rather than crash. If a TTS sentence contains only citation markup and punctuation, the playback pipeline should handle the empty sanitized text. A tiny silence WAV became the solution for that last case, not because silence has semantic value, but because the audio transport needed a valid buffer to continue across a transition.

This was an important change in engineering posture. The goal was no longer to create a demo path that worked under ideal circumstances. The goal was to create a system whose failure modes could be detected, described, recovered from, and tested.

PART IV: THE LIBRARY GETS A NAME AND A SENSORY WORLD

13. From Smart Reader to Omnitome

A product name should not decide the architecture, but the architecture can reveal when a name has become too small.

Smart Reader described an AI-assisted reading experience. It was understandable, but generic. It also carried residue from an early manga-reader framing even after the implementation had grown to handle textbooks, web material, research sources, and multilingual documents. The internal package name and documentation did not always agree with the new direction. The project needed a name that could hold a local reading library, a compressed representation, audio, a source archive, and the possibility of later formats without promising that every future idea already existed.

The working name **Omnitome** emerged in the September 21 to 23 period and became the user-facing identity through the subsequent 5.8 rebrand work. The public repository stayed named `Smart-Reader` for part of that transition. Later, the repository itself was renamed to `Omnitome`, and the root package metadata was updated so the application no longer described itself with the old manga-reader phrase. That delay between product name and repository name is ordinary, but it illustrates the number of surfaces a rebrand touches: package files, window title, theme tokens, readme, app icon, synthesis-title language, docs, installer metadata, and internal naming rules.

The name changed; the invariant did not. If anything, the rebrand made the product's central promise easier to say. Omnitome is not an AI chatbot that happens to open a book. It is a local-first neural reading library with provenance-aware semantic compression. The app's identity is the reading workflow, and the source remains its proof.

Five worlds, one reading instrument

The visual system grew alongside the product name. An early dark-first direction was called `neon`, but the chosen identity became **Omni**, a deep-violet world with glowing purple accents and sigil-like motifs. From there the theme architecture evolved into five adaptive themes:

- **Omni:** the signature deep-violet, cosmic identity.
- **Sakura:** a gentle palette with soft pink accents and editorial typography.
- **Spring:** sage greens, calm contrast, and generous whitespace.
- **Coffee:** warm sepia and amber, with an old-library atmosphere.
- **Cyberpunk:** a high-contrast dark palette with cyan and magenta illumination.

The themes were not intended to change the product's underlying meaning. The Original view retained neutral, paper-like typography. The derived Omni lens carried a restrained theme tint. That was a deliberate division of labor: the source should read as evidence, not as a visual effect. The application could be expressive without turning every page into a poster.

Theme engineering had its own evolution. CSS was modularized, accent tokens were calibrated per theme, decorative elements were adjusted, and scrollbars became responsive to theme colors. Procedural SVG cover art used stable data such as title hashes to give entries identity without requiring a downloaded cover image for every text. “Source Only” and “Empty” badges distinguished books that had been imported but had not been compressed into the regular reading experience. KaTeX supplied mathematical typesetting where source content included formulae.

These choices may sound cosmetic compared with embeddings or database lifecycle. But an application for long reading is partly a place where a person spends time. The visual system affects whether that place feels coherent, whether source and derivation are distinct, and whether the reader can remain there without fighting the interface.

The difference between atmosphere and distraction

The theme work also taught a boundary. A reading product can feel crafted without coating every control in animation and glowing decoration. The Omnitome identity was meant to be recognizable and a little magical, but the design principle was restraint. The lens could be tinted; the page still needed to be readable. The theme should support the user’s attention, not compete with it.

That boundary also helped future planning. Animated storytelling, discussion, and more elaborate audiovisual representations remained possible directions, but they were not necessary to make the v1.0 reading product coherent. The immediate work was to make one experience feel finished, not to make every imagined future mode partially visible in the navigation.

By the time the app moved toward native desktop packaging, this visual identity was no longer a loose set of mockups. It had a five-theme architecture, a book emblem, procedural covers, standardized spacing and color variables, and an interaction language that could grow with the core system. The brand was an interface design constraint, not a substitute for function.

14. A Voice That Can Stay Offline

Audio began as a planned expansion, then became one of the most technically demanding elements of the finished application. Reading can be silent and still benefit from semantic compression. But a local voice layer offered another way to experience long material: let the application narrate the current chapter, highlight the sentence being read, scroll as the voice advances, and eventually move from chapter to chapter without requiring constant mouse input.

The project chose **Kokoro-82M** for local neural text-to-speech. Rather than asking a remote speech service to generate every sentence, Omnitome could run the voice model through its Python sidecar. That aligned with the local-first direction and gave the app control over the audio cache and playback lifecycle. Four curated voices were integrated and verified in the mature setup: `af_heart`, `am_adam`, `bf_alice`, and `bm_george`. The settings interface offered voice and speed choices, auto-scroll behavior, cache management, and a keep-loaded setting.

The TTS pipeline required much more than a call to a `synthesize` function. The backend client coordinated sidecar readiness, sentence-level requests, deterministic sentence hashes, a two-level sharded audio cache, and atomic storage. The frontend managed play, pause, seek, sentence highlighting, and navigation across the text. The system needed adjacent sentence audio ready before it was requested so that playback could remain smooth. A dual-deck approach allowed one segment to play while the next was prepared, reducing gaps and making long-form narration feel continuous.

The UI also had to distinguish voice synthesis from reading progress. If the current sentence is highlighted, the text position and audio position should agree. If the user manually scrolls, auto-follow must not behave as though the user's hands do not exist. When a chapter ends, a podcast-style auto-advance mode can move to the next chapter, but only if queue state and player state remain synchronized.

One technical challenge was resource use. The sidecar might have already loaded BGE-M3 or the reranker. Loading Kokoro could compete for the same GPU memory. The lifecycle manager therefore gained an idle-eviction strategy that unloads expensive models after inactivity; the documented TTS path describes Kokoro weights being evicted after five minutes, with reload on demand. This introduced a trade-off between immediate playback and available memory. It is the same resource-lifecycle problem seen in semantic indexing, now attached to the user's ears.

An offline promise requires more than downloaded weights

A model is not truly available offline simply because its large weight file exists on disk. A library may attempt to resolve voices from a remote model hub. A phonemizer may expect separate dictionaries and native libraries. A dependency may try to download a language model the first time it initializes. Python may be able to load a package in a development environment only because the environment already contains a dependency that was never explicitly bundled.

The project encountered these cases one by one. The Kokoro voice loader initially called a Hugging Face download helper even when voice files were present locally. Offline mode then raised `OfflineModeIsEnabled` instead of using those files. The fix introduced a local voice-path

resolver that checked the model directory, application model paths, repository resources, and local Hugging Face snapshots. It could copy a found voice tensor into the model directory for future lookups and load the local tensor directly.

A separate edge case involved text made entirely of citation backlinks or punctuation, such as a generated sentence containing only `[p12]`. Sanitization removed the markup and left an empty string. The synthesis function quite reasonably objected to empty text, but the error broke the player's transition between sentences. The repair returned a valid 250-millisecond mono, 24 kHz, 16-bit PCM WAV containing silence when the cleaned text was empty. Silence here is a transport-level placeholder. It lets the player preserve sequence without trying to pronounce a citation marker or crashing between meaningful sentences.

The project also added a `availableVoices` list to its TTS status endpoint and improved error handling in the Python routes and Node client. This allowed the UI to reason about what the local engine could provide, rather than assume the entire voice catalog was present.

Listening became a product surface

The eventual local voice system included sentence highlighting, auto-scroll, chapter-title narration, podcast-style auto-advance, disk caching with least-recently-used eviction, and CPU fallback where supported. The goal was not to make speech a novelty button. It was to make audio a coherent reading mode connected to the same source and derivative structure as the rest of the app.

The most revealing detail is how many layers had to agree. A sentence could be properly synthesized and cached, yet still fail to play due to browser security. A voice tensor could exist locally, yet fail to load because the engine called a network function. The application could speak every sentence, but a queue error could skip one when the user paused and resumed. "TTS works" became meaningful only after the full chain behaved as one experience, including in a frozen, offline installation.

15. Sound as Interface

The sound engine began in a familiar way: temporary Web Audio oscillators could produce cues for clicks and operations. That was enough to establish the interaction concept. It was not the final form the project wanted. The team moved toward a deterministic pipeline that generated a fixed set of audio stems in advance and shipped them with the application.

The finished v1.0.2 sound system documented in the repository contained **90 WAV files**, representing 18 canonical interaction events across five themes. The build pipeline lived in `tools/sound-design/` and used deterministic digital signal processing techniques: frequency modulation, resonant filtering, noise sources, amplitude envelopes, and peak normalization. Each stem was written as mono, 48 kHz, 16-bit PCM and normalized to a peak of approximately -3 dBFS. The output was an asset library, not runtime synthesis.

Why make sound assets ahead of time instead of improvising them every time a button is clicked? Predictability, for one. A deterministic set of rendered files can be reviewed, measured, bundled, and verified before release. It avoids the variability and runtime cost of generating a new sound at the moment the user interacts. Loading the stems into Web Audio buffers allows the system to play short cues without waiting for a slow file read at each event. The sound is part of the interface's timing, so latency matters.

Each theme had its own auditory language. The `omni` palette carried the signature brand sound. `sakura` suggested softer, lighter textures. `spring` used gentle, organic cues. `coffee` leaned into warm, tactile page-like textures. `cyberpunk` used a more electronic vocabulary. The internal sound palette identifiers in the DSP layer did not always share the same names as the visual themes; the important point was the mapping between them and the final theme-specific stems, not pretending that every internal label had always been identical.

The system routed semantic events rather than making components assemble arbitrary audio behavior. Examples included primary or secondary button presses, toggles, navigation between Library, Research, and Import, book-opening transitions, operation starts and completions, warning events, and a theme-switch signature motif. The event map created a vocabulary for audio feedback and a priority model so that important cues were not treated like ambient decoration.

Settings exposed the main sound toggle, volume control, theme-switch sound toggle, and a preview of the active theme motif. The volume slider used a perceptual curve rather than assuming that doubling the numeric setting would sound twice as loud. During active Kokoro narration, non-critical UI sounds were ducked or suppressed so the interface would not chatter over the book.

A small engine with a real specification

The test suite validated all 90 generated files, including file count, WAV headers, encoding, sample rate, channel count, and normalization limits. A separate manifest verification step checked the published assets. The generation pipeline could be rerun and the result compared against the expected design outputs. This is a better fit for a polished desktop product than a set of loosely tuned oscillator snippets scattered across React components.

The audio work also revealed a shared browser constraint. The sound engine and the TTS player could not be treated as completely independent users of audio hardware. A user click can unlock audio playback, whereas an asynchronous request made later may run outside that gesture context. To make the two systems cooperate, the application adopted a shared master `AudioContext`, with sound-player helpers that allowed the TTS player to resume that same context from user interactions.

This mattered especially in WebView2, the browser runtime used by the Windows desktop package. A backend request could return `200 OK` and a valid WAV buffer while the user heard nothing because playback was attempted outside the allowed user-gesture flow. The application needed both the bytes and a permitted playback path.

Sound design might appear far removed from a semantic compression product. In practice, it illustrated the same philosophy in miniature. An interaction should communicate a specific event. The event should map to a defined asset. The asset should be available when needed. Playback should not interfere with narration. Settings should make the system controllable. Tests should verify the deliverables. A tiny sound can be designed with the same rigor as a large pipeline, and the result is a more coherent experience.

PART V: A WEB APPLICATION LEARNS TO BECOME A DESKTOP APPLICATION

16. Why Tauri Changed the Problem

Before the Tauri work, Omnitome was already a sophisticated local application with a React frontend, a Node.js backend, a Python sidecar, a SQLite database, and large model dependencies. The browser-style interface was useful for iterative development, but a polished Store Edition needed a desktop shell that could coordinate the processes, manage startup and shutdown, and provide a consistent application window.

The project selected **Tauri v2**, with Rust serving as the native shell and process supervisor. The frontend stayed in React, Vite, and Tailwind. Node.js with Express remained the application orchestrator and sole SQLite writer. Python with FastAPI remained the compute sidecar. The desktop shell did not replace these responsibilities; it coordinated their lifecycle and provided the packaged environment in which they ran.

That distinction shaped the packaging architecture. A Tauri executable that merely opens a web page is not the same as an application that launches its local backend and neural sidecar reliably. Omnitome had to resolve paths differently in development and in a packaged build, select available ports, pass those ports to the frontend, start child processes, detect readiness, and terminate the processes when the user closed the window or the application exited.

S11 established the Tauri v2 scaffold. It verified that Rust, Cargo, the Microsoft C++ build tools, and the Tauri CLI were available, and launched an Omnitome-titled development window. S12 developed the Python sidecar freeze, Node/Python startup flow, dynamic ports, and Rust supervisor. S12c then required deeper cleanup after testing showed the sidecar could survive a window close. The fix used process-tree termination on Windows and added close-request and application-exit hooks so the supervisor would clean up both Node and Python.

A native wrapper changes the meaning of “the app closed.” In a browser tab, a developer can often ignore an orphaned local process during a short experiment. In a packaged desktop product, a window close that leaves a large Python process running is a resource and user-trust failure. The supervisor must own that lifecycle explicitly.

Cold start was a product problem

The first startup architecture also tried to initialize too many models too early. On a 4 GB RTX 3050, simultaneously warming BGE-M3, OCR, reranking, and TTS caused cold-start timeouts and VRAM contention. The fix was to warm only BGE-M3 on startup and load other models lazily when first requested. TTS audio elements used `preload="none"` to avoid triggering model-related costs simply because a reader component mounted. The Python startup warmup became a soft-fail operation rather than a fatal gate, and the sidecar timeout was raised to five minutes for operations that could legitimately take longer on a consumer machine.

This was a significant refinement in the product's concept of readiness. A process being alive is not identical to every model being warm. A model not being warm is not necessarily a fatal error. A neural feature can be loaded on demand, provided the UI exposes that wait sensibly and the backend has stable readiness and unavailable states.

The Rust shell needed to remain responsive during startup. When first-run extraction could take one or two minutes, doing that work on the UI thread caused the window to appear frozen. The supervisor setup moved to a background thread, while the splash screen remained visible and animated. The splash was not merely decorative; it made a long, unavoidable initialization phase legible to the user.

The process boundary became a product boundary

By v1.0.2, the desktop topology had a clear division of responsibility:

- **Tauri/Rust:** native window lifecycle, process supervision, startup coordination, and child-process cleanup.
- **Node.js 22 / Express 5:** API routes, job queue, orchestration, model-provider routing, lifecycle logic, and all SQLite writes.
- **Python FastAPI sidecar:** BGE-M3 embeddings, BGE reranking, OCR and parsing support, and Kokoro voice synthesis.
- **React/Vite/Tailwind:** Library, Research, Import, reader, settings, and user-facing state.
- **SQLite in WAL mode:** durable local application data, with Node as the sole writer.

This architecture had more moving parts than a monolithic executable. That was a real cost. It was also a practical division of work: Rust handled native lifecycle, Node handled application state and orchestration, and Python carried a mature ecosystem for model inference and document processing. The system's challenge was to make that division invisible to the reader when things worked and diagnosable when they did not.

17. The Installer Is Another Runtime

Packaging is where assumptions go to be cross-examined.

In development, a Python virtual environment may already contain a library that was installed months earlier. Node may have a full `node_modules` tree including development packages. The model cache may already be populated. A path can be resolved relative to the repository. A port can be hard-coded. The application may be able to reach the network. The GPU driver and CUDA libraries are present on the builder’s machine. None of those conditions should be assumed for a clean installation.

The Tauri installer work forced the team to define what the product actually included and what it needed to acquire on first run. A frozen Python sidecar packaged with PyInstaller had grown to several gigabytes. The Tauri NSIS bundler ran into the two-gigabyte limit of the 32-bit `makensis` toolchain when asked to process the large sidecar payload directly. The workaround was to pre-compress the sidecar as `sidecar.7z` using LZMA, producing an archive reported at about 1.36 GB. The Rust supervisor could then extract the archive into `%LOCALAPPDATA%\Omnitome\sidecar\` on first boot.

The final installer was reported at approximately 1.66 GB. The distinction between that number, the compressed archive, and the full frozen sidecar directory matters. They are different artifacts measured at different points in the packaging pipeline. A product page should use the size of the actual distributed installer, not the sum of loosely related intermediate build folders.

The installer build was also slow and memory-intensive. NSIS compression was switched to store mode to prevent a build that could take hours and exhaust the host machine. Instead of copying an oversized Node runtime, the build script ran `npm install --omit=dev` in a staging directory at `resources/backend-runtime`, reducing that backend runtime from roughly 1.2 GB to around 80 MB in the recorded build. Every such reduction had to be checked against actual packaged startup, because a small distribution that omitted a necessary dependency would not count as an optimization.

First-run model management

The neural models raised another distribution question. Bundling every large model inside the main installer could inflate downloads and make updates cumbersome. The S13 Resource Manager gave models a separate managed lifecycle. It checked available disk space, streamed archives, reported progress through server-sent events, extracted into staging directories, and promoted the results atomically only after successful completion. A lock manifest stored SHA-256 hashes after the first successful download so that later checks could verify integrity. The Python sidecar was configured for offline operation through `TRANSFORMERS_OFFLINE=1` and `local_files_only=True`, with controlled 503 responses if models were missing rather than attempting surprise downloads.

The first-run screen listed the local models needed by the application and showed progress, status, and errors. The BGE-M3 archive, about 1.32 GB in a tar.gz test package when symlinks were dereferenced, was served locally, downloaded, hash-checked, extracted, and promoted to Installed. That exercise validated the pipeline without relying on a cloud transfer to make the demonstration work.

A lower-level Node 22 issue surfaced during streaming: an `undici` assertion crash involving `assert(!this.paused)`. The downloader moved to a manual `getReader()` loop and added error handling around unexpected process-level exceptions. A `MaxListenersExceededWarning` was then resolved by using `events.once` instead of accumulating listener registrations. These were not glamorous features, but without them a model download could die in a way that undermined the entire first-run experience.

Atomicity is a user feature

Staging and atomic promotion are often discussed as deployment mechanics. For a reader whose models may consume gigabytes of disk, they are user-protection mechanics. The application should not announce that a model is installed if extraction stopped halfway through. It should not overwrite a working model with a partial archive. It should not require the user to manually guess which directory contains an incomplete download.

The resource manager's job was to make the model lifecycle legible and recoverable: enough space available, download progressing, checksum verified, archive extracted, model promoted, runtime able to find the files. Each transition had a meaning. An error should halt promotion and leave the installed state intact wherever possible.

Packaging expanded the system from "a repository that runs on the developer's machine" into "a product that has to establish a known state on somebody else's machine." That is an architectural change, not a final button click.

18. The Last Mile: State, Audio, and Frozen Dependencies

A release candidate can be nearly complete and still hide a family of failures that only appear in the exact environment users will receive. The final Omnitome push consisted of these last-mile corrections: path handling on Windows, Python package-data collection, CUDA library bundling, offline voice loading, audio playback restrictions, onboarding state, and stale UI data.

Windows path prefixes

One bug appeared as `EISDIR: illegal operation on a directory, lstat 'C:'`. The underlying problem was associated with a Windows verbatim path prefix such as `\\?\C:\...` passing into Node 22 filesystem resolution. In that reported environment, path normalization could reduce the path to `C:` and invoke an invalid `lstat` operation. The supervisor added a `strip_windows_verbatim` helper to sanitize paths before they crossed into Node or another child process, decoupled some resolution from `resource_dir()` by using the executable's parent directory, and introduced root guards. Dev-mode path resolution was also repaired so the app checked both `src-tauri` and the repository root for the Python virtual environment and backend runtime.

The important verification point was that the fix did not remain a one-off manual tweak. The Rust unit tests were expanded, with all seventeen passing in the recorded S14 closeout. The app needed to resolve paths in a development repository and in a packaged installation, which are related but not identical directory layouts.

PyInstaller and the phonemizer's hidden files

Kokoro's synthesis stack depended on packages that carry runtime data as well as Python code. In the frozen executable, `espeakng_loader` failed because its `espeak-ng-data` dictionary tree was missing. The `.spec` file was updated to collect the package data and metadata and include its dynamic libraries, including `espeak-ng.dll`.

Then `misaki`, a dependency in the Kokoro path, tried to initialize spaCy's `en_core_web_sm` model. On a development environment the model could already be installed, hiding the problem. In the frozen executable, spaCy attempted a dynamic download and failed because the bundled process did not have the expected `pip` or `uv` package manager. The fix installed the model into the sidecar environment and explicitly collected its data files, binaries, hidden imports, and spaCy submodules so the needed factories were registered at startup. The requirements file gained the model dependency and explicit distribution fallback information.

This is a classic "works on my machine" problem, except it happens inside a neural inference package rather than a small web app. Packaging must include the dependency closure that runtime behavior actually needs, not just the imports that a static scanner can see.

CUDA and cuDNN DLLs

The next frozen-build failure looked like a fatal missing-library problem involving `cudnn_engines_precompiled64_9.dll` or one of its dependencies. Collecting a handful of PyTorch DLLs was not enough. The application had to include the relevant Torch resources, hidden

imports, and the DLLs installed through the NVIDIA namespace packages. The eventual change used PyInstaller's `collect_all('torch')` and explicitly traversed the available NVIDIA package paths to gather CUDA binaries into the frozen executable's internal directory, removing duplicate hidden-import entries before analysis.

The result was verified on the packaged sidecar rather than inferred from a successful Python virtual-environment run. The test target was no longer "can Python import torch?" It was "can the actual frozen application initialize Kokoro and use CUDA after installation?"

WebView2 can say yes while the speakers say nothing

The audio path had a final class of bug. The Node client returned a valid WAV payload and a 200 response, yet the frontend remained silent. Pause and resume could skip sentences or play silence. WebView2's autoplay policy restricted playback initiated asynchronously outside the direct user gesture stack. A silent base64 audio element used to prime playback did not leave the application with a usable active deck once it resumed.

The solution was to share the master AudioContext between UI sounds and TTS, resume it directly inside user interactions such as Play and Seek, and decode WAV array buffers into native `AudioBuffer` objects. The main path used `AudioBufferSourceNode` for gapless playback, while an `HTMLAudioElement` fallback was guarded by a check that the payload contained real audio. The TTS system also marked narration as active so low-priority interaction cues could be ducked while a sentence was spoken.

The defect is an excellent illustration of what a production-ready claim has to mean. The model can work, the server can answer, the bytes can be valid, and the user can still hear nothing. The only reliable verification is end-to-end playback in the runtime that ships.

Data that arrives late

The final user-experience fixes were smaller but equally concrete. A completed import could momentarily trigger a false "This book has been deleted" toast because the client cache had not caught up with the backend. The queue manager began treating very recent completed tasks as fresh, refetching the books query and navigating to the book route rather than aborting the action. Ghost tasks referring to previously deleted book IDs were pruned from SQLite.

The Library's lifecycle reconciliation also gained event-driven refresh. A completed queue task emitted a books-changed event, and the Library subscribed and refetched on mount. `refetchOnMount: 'always'` helped prevent stale catalog data when navigating between views. These fixes did not change the semantic model; they made the state already achieved by the semantic model visible without delay.

The content security boundary

The desktop configuration also moved from a null content security policy to a restrictive policy that defaulted to self-hosted resources and whitelisted the local HTTP and WebSocket addresses needed by the child processes. Styles retained an inline allowance where required by the interface. Image data and local dynamic connections were explicitly bounded. The point was to define what the packaged frontend needed instead of leaving the browser layer with an unspecified policy.

This last-mile work has a common pattern. The failure is often a mismatch between two components that each appear correct in isolation: a path format crossing a process boundary, an expected package file that the bundler omitted, an asynchronous audio request crossing a browser policy, or a backend state crossing a stale client cache. Final polish is the work of making those boundaries explicit and testable.

PART VI: WHAT V1.0.2 MEANS

19. A Release Is a Set of Verified Boundaries

The v1.0 line was not one giant release event. It was a sequence of closures. Phase 5.8 consolidated much of the reading surface, provenance, themes, and pre-desktop readiness. Phase 5.9 brought Kokoro TTS through the Python endpoint, Node client and cache, reader controls, settings, cold-start behavior, and podcast-style auto-advance. Series 4 moved the application into Tauri, then developed the frozen sidecar, process supervisor, model resource manager, and installer architecture. v1.0.1 addressed frozen phonemizer and spaCy dependencies, fullscreen controls, queue behavior, and UI polish. v1.0.2 brought together CUDA bundling, the shared audio path, the thematic sound system, and the release-level cleanup recorded on October 22.

The repository handoff at the v1.0.2 milestone described the current HEAD as `77c5906`, with a clean and synchronized tree, and stated that the Windows installer had been packaged and verified end-to-end. The public README labels the release v1.0.2 Stable. The milestone matters because it crossed a threshold: the application was no longer only a development environment requiring the contributor to understand how to launch Node, Python, the frontend, and local models separately. It had a Tauri desktop shell, an installer, packaged dependencies, offline TTS support, and a first-run setup route.

The verification record at that point was substantial. It included seventeen Rust/Cargo tests, sixty backend regression suites plus a network suite in the handoff record, a ten-fixture corpus release gate, TypeScript typechecking, Vite production builds, procedural sound generation tests and manifest checks, and targeted TTS and import queue suites. One test log reported all ninety sound stems verified, all seventeen Rust tests green, and the frontend production build complete without TypeScript or Rollup errors.

Those numbers do not mean that the software is bug-free. They mean that a set of explicit gates passed in the recorded state. Their value is that the tests can be named, rerun, and compared against later changes. A stable release is not the absence of all possible defects. It is the point at which a product’s core promises have been implemented, the known critical failure modes have been addressed to the release standard, and the validation evidence is strong enough to ship the defined scope.

Why the version boundary mattered

It would have been possible to continue expanding Omnitome before declaring a stable release. A richer onboarding wizard, a prebuilt archive of public-domain works, additional packaging polish, a formal Store Edition distribution pipeline, and funding options were already visible as next priorities. The team instead needed to define what was in the release and what belonged to the following productization phase.

This is not a bureaucratic preference for version numbers. It protects product coherence. When mobile, cloud sync, multiple users, payment infrastructure, Story mode, Discussion mode, and background music remain in the roadmap, they can be discussed honestly as future work rather

than quietly treated as v1.0 behavior. A release can be evaluated against a finite promise. Scope control also gives engineering a chance to verify the whole product instead of endlessly adding features to an unstable foundation.

The Store Edition aspiration added a second reason to be precise. A commercial package needs more than code that works for its original builder. It needs a predictable installer, transparent model setup, rights-aware starter content, meaningful onboarding, and a clear distinction between community distribution and commercial distribution. The v1.0.2 release established the desktop foundation. The next phase would need to turn that foundation into a more complete distribution experience.

The milestone in plain terms

At v1.0.2, Omnitome could import supported files and web material, build a local semantic index, create compressed reading units through a configured generation path, preserve a route to original sources, manage a local library and research collection, narrate with Kokoro TTS, emit deterministic theme-aware UI sounds, and run inside a packaged Windows desktop application. Its architecture explicitly separated frontend, orchestration, compute-heavy sidecar work, and local persistence.

That is a considerable distance from a local prompt experiment. But the release is not proof that neural compression can never lose nuance, that the model will perform equally on every device, or that all possible documents will parse perfectly. It is proof of a concrete application boundary that exists and can be tested.

20. The Numbers, and What They Do Not Mean

The benchmark table is one of the most compelling pages in the Omnitome repository. It also needs careful reading.

The documented machine was a Windows 11 laptop with a Ryzen 7 4800H, 16 GB of system memory, an RTX 3050 with 4 GB of VRAM, Node.js 22, Python 3.10 or later, and SQLite in WAL mode. On this hardware, the recorded corpus benchmark included a 508,589-word Machine Learning Systems document, which parsed and chunked into twenty-seven chapters and 2,817 chunks. The end-to-end import for a roughly 508K-word book was later highlighted at about 5.9 minutes. That is a measured result for that document and that system, not a forecast for all 508K-word PDFs or every laptop.

The benchmark notes also recorded around 296 MB peak Node memory during ingestion after the await-barrier change, about 1.1 GB active VRAM for BGE-M3 FP16 in the optimized path, approximately 174 MB of idle VRAM after model eviction, and sub-second processing for a sixteen-chunk embedding batch under the optimized measurement. The document corpus described twenty-two files totaling about 211.5 MB and roughly 2.46 million words across multiple languages. The release corpus had ten fixtures across five formats, with its own assertion classes. These figures tell a story of deliberately measured improvements, not an abstract claim that the product is fast.

The sound benchmarks tell a similar story at another scale: the deterministic generation pipeline produced ninety audio files, and the test process validated their headers and normalization. That is a stronger statement than “the interface makes sound on the developer’s computer.” It describes a reproducible artifact set with properties that can be checked.

But numbers can be made misleading by loss of context. “Thirty times faster” is meaningful only relative to a stated earlier path and workload. “Under one second” is meaningful only alongside the batch size, model configuration, and machine. “Zero false timeouts” in a test population is not a mathematical guarantee that no future large chapter can exceed a timeout. “Seven-to-one compression” can be an intended target or a marketing shorthand without being the mean of a measured corpus. The memoir must keep these categories separate.

What a mature benchmark needs

A good benchmark record should identify the input, the machine, the code path, the unit being measured, and the success criteria. It should be possible to ask whether the comparison used equivalent workloads, whether the old path had a different batch size, whether GPU warmup was included, and whether a timeout or fallback counted as success. If the project changes its compression contract, the older performance figures should not be carried forward as though the implementation remained identical.

The historical review of compression made this especially clear. Some project materials describe approximately 7:1 compression. A review of three cited examples measured 3.61:1, 4.09:1, and 5.05:1, averaging about 4.25:1. Those observations do not prove that the target is impossible. They also do not justify relabeling the observed mean as 7:1. The correct next step is to define a

canonical measurement procedure, re-run it against the current release candidate and representative corpus, and publish the distribution with both the intended target and the observed results.

The same honesty applies to local-first claims. Local embeddings and local TTS were part of the application. But during the earlier hybrid phase, synthesis used OpenRouter with DeepSeek V4 Flash. The privacy boundary therefore depended on configuration. If the user selected a cloud provider, the relevant content sent for generation could leave the device. The later local provider path gave users the option to configure Ollama or LM Studio through an OpenAI-compatible local base URL. The accurate claim is about which operations are local under which configuration, not a blanket statement that every historical build had zero network egress.

Measurement as product ethics

The project's evidence discipline grew partly from ordinary debugging, but it also has an ethical dimension. Users make decisions about whether to trust an AI-generated reading, whether to rely on it for study, whether to delete an original file, or whether to leave a document in a local application based on what the product tells them. An inflated performance claim may disappoint them. An overbroad privacy promise may expose material in a way they did not expect. A citation badge that implies stronger verification than it provides may encourage overconfidence.

Measurements should explain what was measured and what remains unknown. Product claims should be precise enough that an informed user can tell the difference between an architecture feature, a tested guarantee, an empirical benchmark, and a future target. This is not timid marketing. It is a stronger basis for trust than treating every favorable measurement as a universal promise.

21. The Deliberate Unfinished

A technical memoir tends to make a product's future seem like a natural continuation of the current implementation. Omnitome's own logs argue for a more disciplined reading. The vision was broader than v1.0, but the release scope had to be narrower.

The original story-oriented motivation naturally suggested more than text compression. Audio could give the reader an alternative mode of experiencing a long work. More structured story analysis could potentially support character sheets, scene understanding, or illustrated adaptations. A discussion mode could let readers question a passage in the context of their own library. A prebuilt Synthetic Archive could ship with original examples that demonstrate import, chapter slicing, technical content, and OCR robustness without asking a new user to find a suitable document first.

Yet these features have different costs and risks. Turning text into a discussion requires carefully grounding responses to source material. Creating images or video raises new content provenance and rights questions. A public starter library needs original, rights-cleared texts and a repeatable generation pipeline. A Store Edition needs code signing, distribution requirements, model acquisition, dependency management, and a supportable upgrade path. None should be treated as a small extension simply because it can be described in one sentence.

The v1.0 feature freeze made that boundary explicit. Mobile support, cloud sync, multi-user operation, subscription and payment infrastructure, Story and Discussion modules, and ambient background music remained outside the v1.0 scope unless they were deliberately re-approved. The priorities were release blockers, tested behavior, and a reliable desktop package. The later v1.1 productization plan included welcome-wizard polish, model setup guidance, generation of offline reference archives, a distribution strategy distinguishing the Community Edition from the commercial Store Edition, and optional funding configuration.

The current repository history by the time this manuscript was prepared already showed the beginning of that next phase, including work labeled as a v1.1 Welcome Wizard, SoundStore, and Theme Catalog. That development belongs after the v1.0.2 boundary rather than inside its feature claims. It is evidence of continuation, not a reason to retroactively rename the earlier milestone.

Open problems are not all defects

Some unfinished items are bugs: a known false toast, an offline model lookup that still calls a remote resolver, a missing dynamic library, or a queue state that never reconciles. These should be fixed or accurately listed as release limitations. Other unfinished items are deliberate absences: no Story mode in v1.0, no payment flow, no cloud sync. They reflect a choice about what the release promises.

A third category is a research uncertainty. How should fidelity be measured across narrative fiction and technical texts? What is the right evaluation suite for "preserving the important structure"? When does a short derived chapter become too compressed to preserve character development or the caveats in an argument? How should retrieval report a partial or indirect

source match? Which local model is a good balance among quality, memory, context length, latency, and license? These cannot be resolved by polishing a button. They need evaluation, comparison, and iteration.

The distinction helps prevent a roadmap from becoming a punishment. A product vision can contain ambitious future directions without pretending those directions are ready. The goal is to make the current promise robust and to let future work build on stable boundaries.

When a test fixture becomes a rights problem

The project also learned that a successful parser test can create a different kind of risk. A document may be excellent for debugging because it is large, realistic, and full of structural edge cases. That does not automatically make it suitable for redistribution in a public repository or a commercial starter library. A fixture can be technically useful and still carry terms that are incompatible with the way the product is being shared.

The saved project history records a licensing transition. The public repository moved to **AGPL-3.0-only**, old MIT release tags were removed, and the project documented the MIT-to-AGPL boundary in `README.md` and `RIGHTS.md`. The developer also cloned the repository into an `Omnitome-Private` line under MIT for possible proprietary v2.0 work. That split should not be interpreted as a general statement about what a license permits in every situation; licensing choices require context, and users should consult the actual license files and competent legal advice for consequential decisions. It does show that the project began to distinguish the license of the software, the intended distribution channel, and the rights attached to the content used to test it.

The saved Chat 14 handoff also records a `git-filter-repo` history rewrite, reportedly touching around 120 commits, to purge fixtures that were not suitable for non-commercially restricted redistribution. That operation rewrote historical commit identities. As a result, older session logs can contain commit hashes that no longer correspond to the rewritten public history. The safe practice was to consult the current `git log` rather than treating an old handoff's SHA as a permanent identifier.

This is not merely repository housekeeping. A Synthetic Archive needs original or clearly rights-cleared material. It should be varied enough to exercise parsing and synthesis, but should not use an attractive copyrighted novel as a convenient demonstration asset unless the rights actually allow that distribution. Technical reproducibility and legal reproducibility intersect: a fixture is most useful when a future developer can run it, inspect the result, and redistribute it under terms that are understood.

The lesson is broader than Omnitome. Test datasets, sample PDFs, documentation screenshots, model weights, fonts, and generated assets all bring their own provenance. A product can meticulously preserve the provenance of sentences while neglecting the rights provenance of the files it ships. The second form of provenance is just as important when the project leaves a private development environment.

The Synthetic Archive as an engineering instrument

The Synthetic Archive concept arose as a way to demonstrate the product with a small library of original, copyright-safe books. In this role, the books are more than sample content. They can exercise the exact behavior Omnitome is designed to handle: a long-form narrative, a procedural or technical document, structured chapters, citations, and documents intentionally varied in format. A clean PDF and a degraded OCR test variant can reveal whether the pipeline behaves differently when text extraction is reliable versus when it must infer text from a page image.

That archive needs more care than generating several tiny placeholder files. If the content is too short, it does not exercise chapter slicing or long-document behavior. If all items have the same structure, they fail to expose parser assumptions. If a source is not copyright-safe, the archive creates licensing risk instead of reducing friction. If a book claims to document Omnitome, it should not invent functionality or history. The Field Guide and this memoir therefore have two roles: they are reading material for users, and they can become real-world fixtures for testing import, parsing, synthesis, and provenance.

A synthetic archive can make the product easier to understand. It can also become a regression corpus if the content remains stable and the expected properties are defined. That is a more durable contribution than a carousel of marketing screenshots.

22. Rules Learned the Expensive Way

The project left behind a set of engineering rules that are more general than Omnitome. They were earned through the particular combination of a growing frontend, a local model stack, a long-running import pipeline, and desktop packaging, but they apply to many AI applications.

1. The source must outlive the transformation

If the product derives content from a source, retain a clean conceptual boundary between them. Never let the output silently replace the input. Keep enough metadata and provenance to navigate back. If the source is detached or removed, represent that state explicitly. This rule shaped the Two-Representation Invariant and the lifecycle state model.

2. A prompt is not a specification

Prompts can guide models, but requirements that matter to the product need code-level checks. Length boundaries, job states, source mappings, parsing contracts, duplicate detection, and lifecycle transitions need tests. The model's response is output to be evaluated, not the definition of success.

3. Backpressure belongs in the architecture

Unbounded concurrency can turn an ordinary desktop workload into a memory disaster. The Node await barrier reduced peak ingestion memory dramatically in the documented benchmark. The broader rule is to model queue depth, bounded batches, sequence length, timeouts, and model residency deliberately. A work queue that respects the machine can outperform a theoretically parallel design that exhausts it.

4. Models have lifecycles

Loading is not free, and keeping a model resident is not free. Cold-start cost, VRAM, disk space, CPU fallback, idle eviction, offline weight availability, and voice-cache size are product concerns. They belong in UI settings and resource management, not only in a model wrapper.

5. One authoritative writer simplifies state

The architecture assigns all SQLite writes to Node. Python provides compute and returns results; React presents state; Rust supervises processes. That reduces the chance of multiple components independently mutating the source of truth. Clear ownership makes debugging and recovery more tractable.

6. An integration boundary is where many failures hide

Node and Python can each pass isolated tests while their shared request contract is wrong. Backend and frontend can each look reasonable while one uses `book_id` and the other expects `bookId`. A frozen executable can fail although the virtual environment works. The correct verification unit is often the full interaction between components.

7. A green suite must be named

"Tests passed" is too vague. Which tests? Unit suites? Integration tests? Corpus gate? Rust supervisor? Typecheck? Production build? Manual launch? Clean installation? A trustworthy report lists the exact gates and their results. This allows a later engineer to know which claims have evidence and which remain assumptions.

8. A successful response is not a successful experience

A TTS endpoint returning HTTP 200 did not mean the user heard anything. A completed ingestion job did not mean the Library displayed it immediately. A model download finishing did not mean it was safe to promote until the checksum passed. The system should be verified at the point where the user receives the result.

9. Fail closed where integrity is at stake; recover gracefully where possible

A checksum mismatch should not install a model. A parser that corrupts math should not be enabled merely because it is new. An unavailable model should return a clear controlled response. An empty TTS sentence can return a valid silent buffer because there is no meaningful speech to protect, and the player can continue. Robustness means choosing the response that preserves the right invariant, not always throwing or always pretending to succeed.

10. Separate aspirations from measurements

A target compression ratio, an observed ratio, a benchmark for a particular source, and a marketing summary are different statements. So are "local embeddings," "offline TTS," and "zero network activity for all configurations." The product deserves ambitious goals, but every published claim should say what kind of claim it is.

11. Test with the documents people actually have

A toy paragraph will not reveal an EPUB spine bug. An ordinary UTF-8 novel will not test Shift-JIS. A digital PDF will not exercise OCR. A happy-path chapter will not reveal an over-detection problem in mathematical extraction. A corpus needs diversity and licensing discipline.

12. Keep scope small enough to finish, then finish it

Omnitome's larger vision included audio, story modes, and other ways to experience long works. v1.0 concentrated on a reliable library, source traceability, import, synthesis, and desktop packaging. Completing that narrower system created a better base for later expansion than a broader menu of half-working features would have.

13. AI-assisted engineering still needs engineering judgment

Agents can save time, but they must not become the final arbiter of repository state. Inspect diffs. Verify the active branch. Run tests. Confirm that the expected files actually changed. Watch for stale buffers and generated-file corruption. Do not let an impressive summary substitute for raw evidence.

14. Reproducible assets are better than hidden runtime improvisation

The procedural sound engine demonstrates this clearly. Its ninety files can be generated and validated as assets, then loaded through a predictable playback path. The same idea applies to model archives, package manifests, corpus fixtures, and release builds: generate, verify, record, and ship defined artifacts.

15. The end-to-end path is the product

Omnitome is not BGE-M3 alone, Kokoro alone, React alone, or a citation marker alone. It is the journey from an imported file to an understandable derived reading that remains connected to its source, with a UI, a queue, local model resources, and recovery behavior that make the result usable. The product lives at the boundaries between its parts.

Epilogue: The Lens Does Not Replace the Page

At the beginning, the thought was simple enough to fit into a prompt: take a very long story and help me get through it faster. By v1.0.2, that prompt had become a local-first desktop application with a semantic index, a compression pipeline, source-linked reading, multi-format ingestion, an adaptive theme system, procedural sounds, neural narration, a model resource manager, and a native process supervisor.

The distance between those two points is measured not only in code. It is measured in questions that became more exact.

What do we mean by “important”? How do we tell a conclusion from its supporting steps? How should a local model share a small GPU with the rest of the application? What is the difference between a generated sentence and the source passage it cites? How can a job be truly complete if the frontend still shows stale state? What happens when the model weights exist but the phonemizer dictionary is missing? When does a local app stop being a developer tool and become a product another person can install and trust?

None of these questions had a single answer. The project improved through successive approximations: a weak embedding path gave way to BGE-M3; raw prompts gave way to a compression contract; passive source references gave way to clickable provenance; fake progress gave way to persistent queue state; unrestricted model loading gave way to bounds and idle eviction; a development environment gave way to a frozen sidecar and a Tauri supervisor; temporary sounds gave way to a deterministic audio library; and an HTTP response that was technically valid gave way to actual audible playback.

The release is meaningful because it binds those lessons into a product. It is not the end of the idea. The next phase can improve onboarding, create a usable Synthetic Archive, refine distribution, and investigate more ambitious reading experiences. But those future directions should inherit the same principle that made the application distinctive: transformation should remain accountable to the thing transformed.

A compressed reading can be a better way into a text. It can reduce repetition and bring the main structure into view. It can help the reader spend attention where attention matters. It can provide a voice, a different atmosphere, and a more navigable route through an immense work. Yet it cannot become identical to the source simply by being useful.

The original remains the page. The Omni reading remains a lens. The reader remains the person who decides when to keep moving and when to go back.

That is the engineering achievement behind Omnitome, and also the limit it chooses to respect.

Appendix A: A Condensed Development Timeline

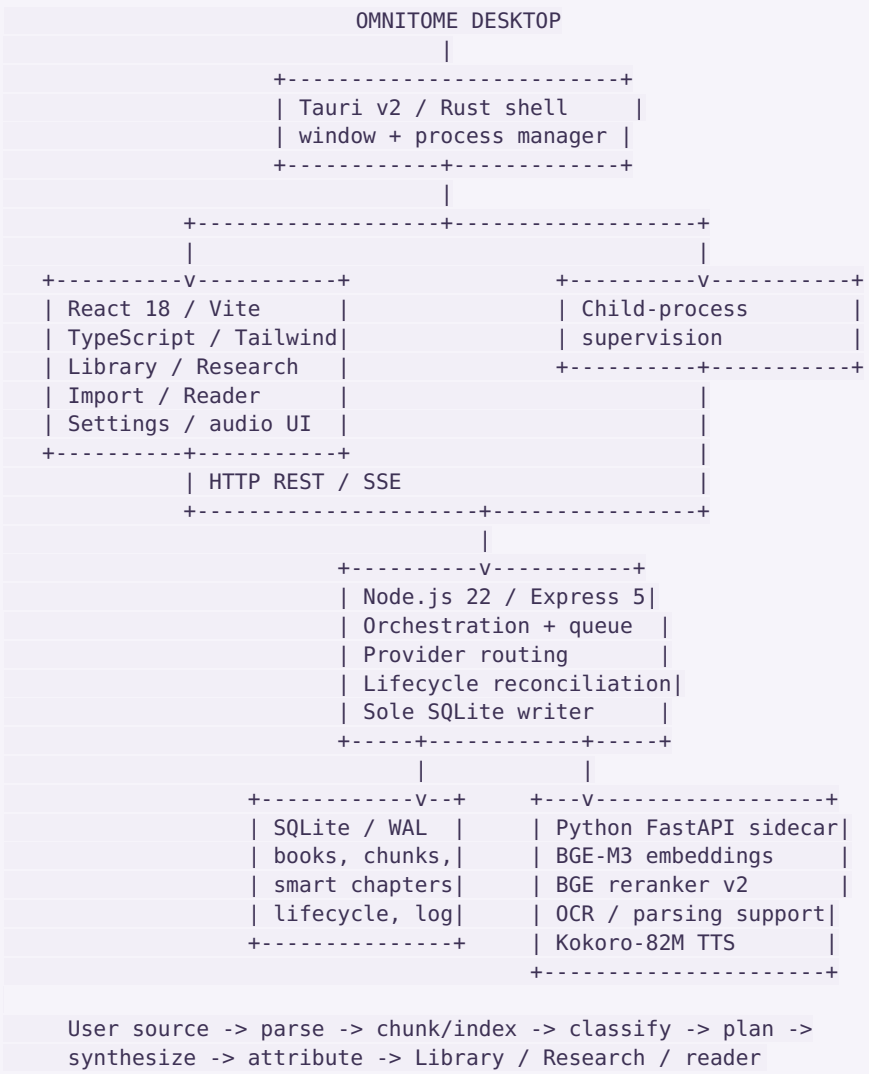
This timeline combines dated session records, repository documentation, and the current release handoff. Early origin details are identified as recollection rather than as a verified commit history.

Period	Development milestone	Why it mattered
Late 2024	Local GPT4All summarizer experiments, as recalled in the project history	Established the personal reading problem and the initial interest in local language-model assistance. Exact model and benchmark details are not preserved in the available source set.
Early Smart Reader phases	Document structure and progressive smart-chapter synthesis evolved through staged phases	Moved the idea beyond a single prompt call into parsing, planning, chunking, synthesis, and reader-facing output.
September 17, 2026	BGE-M3 migration and React frontend architecture work	Replaced the earlier 256-dimensional local vectorizer with 1,024-dimensional BGE-M3 embeddings; CLS pooling was selected after comparison. React/Vite/Tailwind work began replacing the legacy frontend.
September 18, 2026	Reader, Book Details, Smart mode, progressive generation	Connected API data to reading routes and refined how derived chapters appeared in the interface.
September 19-21, 2026	Import progress, queue reliability, compression enforcement, legacy frontend cleanup	Replaced misleading progress assumptions with observable jobs, improved fallback honesty, tightened compression behavior, and removed the vanilla frontend.
September 21-23, 2026	Product identity and architecture reassessment	Introduced Omnitome as working title; clarified Original versus Smart/Omni Reading; confirmed local embeddings alongside cloud generation at that stage.
September 23-25, 2026	Phase 4 reliability closure; Phase 5 provenance foundation and click-through UI	Corrected import trust failures, added bibliographic metadata and deterministic source resolution, and made provenance a reader interaction.
September 26-29, 2026	Library and Research lifecycle; validation loops; Python sidecar maturation	Expanded the product from smart chapters toward a local-first reading library with a clear source catalog and compute boundary.

Period	Development milestone	Why it mattered
Late September 2026	Adaptive compression, chapter parsing, source attribution refinements	Replaced brittle assumptions with bounded, tested contracts and improved handling of source chunks and long documents.
Early October 2026	Omnitome rebrand and five-theme visual system	Standardized product identity, themes, decorative covers, typography, and design laws.
October 8-10, 2026	GPU optimization and import performance investigations	Reduced BGE-M3 memory use with FP16, bounded sequences and batching; addressed excessive Node concurrency, long-document processing, and telemetry.
October 11-15, 2026	Multilingual ingestion, quality validation, Kokoro TTS and release preparation	Improved Japanese/Aozora and other parser edge cases; delivered the local speech pipeline and expanded the test corpus.
October 16-18, 2026	Tauri supervisor, sidecar lifecycle, model resource manager	Made startup, shutdown, model download, integrity checking, and missing-model behavior part of the application design.
October 19, 2026	NSIS installer architecture and splash screen	Avoided the 32-bit `makensis` limit by precompressing the sidecar as 7z; moved extraction off the UI thread.
October 20, 2026	Offline Kokoro voice loading; lifecycle self-healing	Removed hidden network assumptions from voice loading and reconciled imported books into the Library lifecycle.
October 21, 2026	v1.0.1 packaging and UX stabilization	Bundled phonemizer and spaCy assets; fixed queue UX and Research scrolling; introduced the 90-file themed sound system and fullscreen support.
October 22, 2026	v1.0.2 Stable milestone	Completed CUDA/cuDNN bundling, shared AudioContext playback, procedural UI sounds, onboarding behavior, and end-to-end Windows installer verification.
After the v1.0.2 boundary	v1.1 productization work began, including Welcome Wizard and theme/sound-store work in repository history	This is the next chapter of development, not a retroactive feature claim for v1.0.2.

Appendix B: The Architecture at v1.0.2

The following diagram summarizes the responsibilities documented at the v1.0.2 milestone. It is an explanatory map, not a claim that every request follows every subsystem.



Responsibility boundaries

Tauri/Rust owns the native desktop shell, startup and shutdown coordination, and child-process supervision. It should kill the child process tree when the app closes and keep the UI responsive while long extraction work happens in the background.

Node.js/Express owns application orchestration, queue persistence, provider routing, lifecycle coordination, the local HTTP API, and SQLite writes. It serializes the source of truth rather than allowing the frontend and Python sidecar to compete over the database.

Python/FastAPI hosts compute-heavy capabilities and libraries suited to machine learning and document processing. The v1.0.2 path includes BGE-M3 embeddings, BGE Reranker v2, OCR support, parsing helpers, and Kokoro TTS. The sidecar's model lifecycle is constrained so that memory usage remains manageable on consumer hardware.

React/Vite/Tailwind provides the user experience: the Library, Research dossiers, Import tabs and queue, book details, reader, settings, audio controls, themes, and source navigation. It should reflect backend state rather than invent its own version of completion.

SQLite stores the local application data, including books, chapters, smart chapters, semantic chunks, lifecycle information, and associated records. WAL mode supports the application's database workload; Node remains the single writer.

Main transformation path

1. The user submits a file, URL, or pasted text.
2. The import queue persists and reports the job state.
3. The ingestion path extracts and normalizes document text and structure.
4. Semantic indexing divides the source into chunks and computes local embeddings.
5. Classification and editorial planning identify the content type and reading structure.
6. Synthesis produces bounded Omni reading units through the configured generation provider.
7. Retrieval, reranking, and attribution create traceable links back to source passages.
8. Lifecycle reconciliation activates the Library entry if the derived chapters exist.
9. The user reads, searches, listens, or follows a citation back to Research.

Provider configuration and privacy boundary

During the earlier hybrid phase, the documented configuration used OpenRouter with DeepSeek V4 Flash for generation while embedding and retrieval were local. The later settings path supports a cloud OpenRouter provider and an OpenAI-compatible local provider, with presets for Ollama and LM Studio. A local provider can use `http://localhost:11434/v1` for Ollama or `http://localhost:1234/v1` for LM Studio, with the model name matching the model served by that runtime. The exact model must be installed and available in the local runtime. A local base URL is not evidence that the model itself is present or that its context window and quality are adequate.

The privacy statement must therefore follow configuration. With a remote generation provider, the prompt and selected source context sent for generation may leave the device. With local generation, the generation request can remain on the device, provided the configured runtime, model, and relevant dependencies are truly local. The application should not disguise that difference.

Appendix C: Selected Engineering Measurements

These are selected historical project measurements, principally from the repository's benchmark and v1.0.2 handoff. They are not universal performance guarantees.

Metric	Recorded value	Context and caveat
Test machine system RAM	16 GB	Windows 11 laptop with Ryzen 7 4800H and RTX 3050 Laptop GPU.
GPU VRAM	4,096 MiB	Consumer laptop GPU; results reflect this constrained environment.
BGE-M3 active VRAM before optimization	About 3,959 MiB	Recorded historical baseline.
BGE-M3 active VRAM after FP16 optimization	About 1,135 MB	Measured with the optimized model path.
Peak Node ingestion memory before await barrier	About 14.6 GB	Earlier unbounded asynchronous behavior under the measured ingestion workload.
Peak Node ingestion memory after await barrier	About 296 MB	Project-reported optimized measurement; approximately 98 percent lower than the recorded baseline.
Sixteen-chunk embedding batch	About 124 seconds to under one second	Historical comparison after combined batching and model optimization; workload-specific.
Large-book import	About 5.9 minutes	Measured on a 508K-word document on the project machine.
Idle VRAM after model eviction	About 174 MB	Reported after expensive model weights were unloaded.
VRAM returned by idle eviction	About 3.2 GB	Reported across the relevant model lifecycle tests.
Reranker timeout range	8 to 60 seconds	Input-calibrated timeout range in the documented implementation.
Release corpus gate	10 of 10 fixtures	Covers multiple assertion classes and source formats.
Wider document benchmark	22 documents, about 2.46 million words	Corpus aggregate; results vary substantially by file format and layout.
Rust supervisor tests at release	17 passing	Recorded `cargo test` suite.

Metric	Recorded value	Context and caveat
Backend regression suites at release	60 passing	Handoff additionally recorded a network suite and corpus gate.
Procedural sound assets	90 WAV files	18 canonical events across five themes, 48 kHz PCM16 mono, approximately -3 dBFS peak normalization.
NSIS installer	About 1.66 GB	Recorded installer output, not the size of the whole source tree.
Compressed frozen sidecar archive	About 1.36 GB	`sidecar.7z` used to avoid the direct NSIS bundler size limitation.
Backend runtime in staged build	About 80 MB	After excluding development dependencies; earlier staging was reported around 1.2 GB.

Compression figures need separate labels

Some product documentation describes approximately 7:1 semantic compression. A historical review recorded three example ratios of 3.61:1, 4.09:1, and 5.05:1, yielding a mean around 4.25:1. The number of examples is small and should not be represented as a universal distribution. Future public claims should state the measurement method, corpus, range, and date.

Appendix D: An Error Catalogue from the Road to Release

This catalogue collects representative bugs and their engineering meaning. It is not exhaustive, and the descriptions reflect the recorded fixes rather than guarantees that the same failure can never recur.

Symptom	Root cause or boundary	Corrective principle
Library entry did not appear after import	Lifecycle flag initialized before smart chapters existed; no reconciliation occurred later	Reconcile at synthesis completion, queue completion, lifecycle access, and startup backfill.
Progress appeared stuck at 35 percent	UI progress did not follow the true stage progression consistently	Use canonical, monotonically increasing stage weights tied to the workflow.
False “book deleted” toast after completion	Frontend cache lagged behind persisted backend state	Refetch recently completed tasks and navigate based on verified queue state.
Duplicate synopsis generation	Pipeline orchestration invoked the same operation redundantly	Define dependencies and a single authoritative job flow.
`book_id` versus `bookId` bugs	Contract mismatch between layers	Normalize names at boundaries and test the full request path.
Import left phantom processing books	Failed job state was not persisted or boot cleanup was incomplete	Mark failures explicitly and make cleanup idempotent.
Weak source match or fallback after timeout	Fixed timeouts and inadequate semantic reranking for large inputs	Bound inputs, batch embeddings, use cross-encoder reranking, and calibrate timeouts.
High VRAM use	BGE-M3 in full precision and excessive concurrent model residency	FP16 where supported, bounded sequence length, controlled batches, lazy loading, idle eviction.
Node memory reached 14.6 GB	Unbounded async work accumulated during ingestion	Use an await barrier and explicit backpressure between dependent operations.
OCR or math parsing over-detected content	Experimental parser rules were broader than the source patterns justified	Add fixtures, gate risky features, and prefer honest limitations over false positives.
Japanese chapters were misdetected	Encoding, CJK word counting, and Aozora heading conventions differed from Western assumptions	Detect encoding, normalize headings, bound chunks, and verify against a real import.
Offline Kokoro voice loading attempted a network call	Engine voice loader unconditionally used a hub download function	Resolve local voice paths and load local tensors directly.
TTS crashed on citation-only text	Sanitization produced empty text, which the synthesis API rejected	Return a valid short silence buffer for empty/citation-only transitions.

Symptom	Root cause or boundary	Corrective principle
Frozen Kokoro missing `espeak-ng-data`	PyInstaller did not include phonemizer data and dynamic library assets	Collect package data, metadata, and native libraries explicitly.
Frozen spaCy path attempted `pip`/`uv` download	Required `en_core_web_sm` model was not bundled	Install and bundle model data, binaries, and hidden imports.
Frozen Torch missing cuDNN DLL	Partial collection did not include all Torch and NVIDIA CUDA binaries	Collect Torch dependencies comprehensively and map NVIDIA package DLLs into the frozen runtime.
`EISDIR: lstat 'C:'` on Windows	Verbatim path prefix interacted badly with path resolution	Sanitize Windows paths, guard roots, and test both dev and packaged directory layouts.
NSIS packaging exceeded its practical tool limit	Large frozen sidecar passed directly to a 32-bit `makensis` path	Pre-compress the sidecar as 7z and extract it at first boot.
First launch appeared frozen during extraction	Heavy supervisor setup ran on the UI thread	Move setup into a background thread and show a responsive splash screen.
TTS endpoint returned 200 but no audio played	WebView2 autoplay rules and invalid active-deck priming	Share and resume AudioContext from user gestures; decode WAV into AudioBuffer playback.
UI sounds interrupted narration	UI effects and narration did not coordinate priority	Use shared audio state and duck low-priority operational cues while speech is active.
Download process crashed during streaming	Node 22 `undici` stream behavior and listener handling	Use `getReader()` streaming, one-shot listener helpers, and explicit error management.

Appendix E: Glossary

AGPL-3.0-only: The license identified by the public repository for the Omnitome Community Edition at the documented release boundary. It should be read from the current repository and distinguished from the terms governing a separate commercial distribution.

Attribution / provenance: The stored and navigable relationship between derived reading content and supporting source passages. In Omnitome, provenance is meant to be clickable and useful for verification; it is not a guarantee that every paraphrase is logically perfect.

Backpressure: A way to control how much work is allowed to accumulate in a pipeline. Omnitome's await barrier reduced the volume of concurrent Node work and sharply lowered the recorded peak memory use.

BGE-M3: A multilingual embedding model used for local semantic vectorization, producing 1,024-dimensional dense vectors in the documented configuration.

BGE Reranker v2: A cross-encoder used to refine candidate passage rankings after initial semantic retrieval.

Chunk / semantic chunk: A bounded segment of source text stored for embedding, retrieval, synthesis context, and provenance mapping.

CLS pooling: A method of deriving an embedding representation from a model's special classification token output. The project selected CLS over an initial mean-pooling approach after recorded comparison tests.

Cold start: The time and resource cost when a process or model initializes from an unloaded state, rather than running with weights already resident in memory.

FP16: 16-bit floating-point representation. In the project's CUDA path, half precision reduced BGE-M3 memory use on supported hardware.

GPT4All: The local-LLM tooling associated with the earliest late-2024 summarizer experiments as recalled in the project history.

HNSW: Hierarchical Navigable Small World, a graph-based approach commonly used for approximate nearest-neighbor search. Its role in a project must be distinguished from a general vector similarity calculation; historical designs and current implementations should be checked before attributing a particular HNSW index to Omnitome's active path.

Kokoro-82M: The local neural text-to-speech model used by Omnitome's audio reading path.

Local-first: An architecture and product priority that performs core storage and suitable computation on the user's device and makes any external provider route explicit. It is not automatically synonymous with zero network traffic in every configuration.

Loss-bounded semantic compression: Omnitome's intended transformation style: reduce source volume while preserving important concepts, reasoning, argument structure, and narrative continuity, within explicit output bounds and with traceable provenance. The exact observed ratio varies by corpus and measurement method.

NSIS: Nullsoft Scriptable Install System, used to create the Windows installer. The large frozen sidecar required a separate 7z archive because the direct bundling path hit a size limitation.

Omni Reading: The reader-facing derived representation, built from source material through synthesis and intended to offer a more information-dense path through the work.

Original Reading: The source representation preserved as the authoritative reference and displayed through Research-oriented views.

PyInstaller: A tool used to package the Python sidecar and its dependencies into a frozen application distribution.

RAG: Retrieval-augmented generation, a pattern in which relevant source passages are retrieved and supplied to a generation model as context.

SQLite WAL mode: SQLite's write-ahead logging mode, used in the local database configuration. In Omnitome's architecture, Node remains the sole application writer.

Tauri v2: The desktop application framework whose Rust shell supervises the local processes and provides the native window environment.

Two-Representation Invariant: The principle that Original Reading and Omni Reading remain conceptually separate: one is preserved source, and the other is a derived, traceable reading lens.

About the Author: The Omnitome Engineering Team

The Omnitome Engineering Team builds local-first software at the intersection of reading, document processing, semantic retrieval, and applied machine learning. The team's work on Omnitome began with a personal question about how to experience very long works without surrendering their narrative or intellectual structure. That question grew into a system for importing documents, indexing them locally, synthesizing compact reading units, preserving provenance, and presenting both source and derivation through a dedicated desktop application.

The team's development practice combines product iteration with evidence-driven engineering. It treats source preservation, explicit lifecycle states, measurable compression behavior, resource-aware model execution, and end-to-end verification as core parts of the user experience. The result is not a claim that AI can read in place of a person. It is an attempt to build a more accountable instrument for people who want to read deeply across more material than time usually permits.

The Making of Omnitome is both a development history and a record of the limits that shaped the software. It celebrates the progress while preserving the unresolved questions, the imperfect measurements, and the fixes that only appeared after the application was tested as a real product.

Selected Project References

The following public project materials were consulted during preparation. Repository contents and file paths may evolve after this edition.

- Omnitome repository: https://github.com/avengersvstheflash/Omnitome
- `README.md`: product identity, feature summary, architecture overview, release status.
- `docs/ARCHITECTURE.md`: Two-Representation Invariant, process boundaries, data lifecycle, ingestion and retrieval topology.
- `docs/FEATURES.md`: supported ingestion formats, synthesis range, provenance behavior, Kokoro TTS, themes, and procedural audio.
- `docs/BENCHMARKS.md`: benchmark environment, corpus measurements, resource use, and historical optimization results.
- `docs/SESSION_HANDOFF.md`: release boundary at v1.0.2, test status, packaging decisions, and immediate productization priorities.
- `docs/ROADMAP_2026-09.md`: historical decisions and roadmap context, read with care because some claims are explicitly marked as requiring re-verification.
- Saved Smart Reader engineering work-session logs 1 through 13 and the Chat 14 standalone handoff.
- Dated session retrospectives and available October 2026 project chat logs.

Omnitome Publishing

The source is the proof. The lens is the path.